

From Open Weights to Decision Policies: A Controlled Study of Scale, Conversion Method and Task Structure in Language-Model Decision-Making

DeadEye project

Draft built 8 October 2026

Abstract

Small open-weight language models are attractive as decision-makers because they are cheap, fast and can run on hardware their users already own. Yet the same weights can be prompted, scored, probed or fine-tuned into a policy, and how this choice interacts with scale has not been measured under controlled conditions. We introduce DEAD EYE, a benchmark and open harness for this measurement. It has seven seeded decision tasks with known oracles (a multi-armed bandit, a contextual bandit, loan approval under distribution shift, gridworld navigation, tic-tac-toe, blackjack and customer-support triage under a written policy) and six conversion methods: the assistant’s own free reply as the unconverted reference, free-form generation, exact likelihood scoring of legal actions, a linear probe on hidden states, LoRA behavior cloning, and a probe on raw task features that uses no language model. Returns are normalized so that a random policy scores 0 and the oracle 1, and common random numbers pair every comparison. Every measurement was made on a single laptop, an Apple M2 Max with 32 GB of memory, so the study can be repeated end to end without a cluster or a cloud account. On four within-family ladders from 135M to 8B parameters, 28 base and instruct checkpoints, we test seven pre-registered hypotheses with every conversion method the machine can run. Five ablation blocks vary prompting, the reasoning budget, quantization, the amount of adaptation data and the structure of the task, and an exploratory comparison sets our conversions against purpose-built decision models that run on the same laptop. Under generation, normalized score changes by [result pending: [slope b , 95% CI]] per tenfold increase in parameters. Fine-tuned with LoRA, the smallest rung of a ladder matches its largest rung, prompted zero-shot and 12 to 15 times larger, on [result pending: [k]] of seven tasks, [result pending: [but not / and also]] under shift, and likelihood scoring closes [result pending: [Z]]% of the gap between the smallest and largest models. Code, configurations and per-decision logs are public.

Draft. This is the complete draft of the paper with the empirical results still to be measured: every blue “result pending” mark is a number the main sweep will fill in, and every red TODO is an open item. Figures and tables that read “placeholder” are generated from the results by the report tool.

1 Introduction

Language models increasingly act rather than answer. They choose moves in games, calls to tools and steps in workflows, and what they produce is executed instead of read (e.g. [Yao et al., 2023](#); [Liu et al., 2024](#); [Paglieri et al., 2025](#)). Many of these systems run on open weights, and small open-weight models are attractive for them because they are cheaper, faster and can run locally. Anyone building such a system faces a practical question: how small can the model be before its decisions get worse, and what can be done about it when they do?

The literature gives mixed signals. Pre-training loss falls predictably as parameters and training data grow ([Hoffmann et al., 2022](#)), but decision quality need not follow. Some abilities appear abruptly at scale ([Wei, Tay, et al., 2022](#)), although many such jumps turn out to be artifacts of discontinuous metrics ([Schaeffer, Miranda and Koyejo, 2023](#)). Several failures persist as models grow. Language models explore poorly in multi-armed bandits unless the history is summarized for them ([Krishnamurthy et al., 2024](#)). Models from 2B to 27B parameters leave much of the action space untried and often act against their own correct rationale ([Schmied et al., 2026](#)), and they can fail even to exploit what they have already seen ([Harris and Slivkins, 2026](#)). Other results favor small models. Given extra test-time compute, a small model can beat one 14 times its size in FLOPs-matched comparisons, on problems it already solves some of the

time (Snell *et al.*, 2025), and distilling bandit algorithms into smaller models let them explore better than larger models without that help (Nie *et al.*, 2025). Most agent benchmarks evaluate frontier APIs or heterogeneous collections of open models (e.g. Wu *et al.*, 2024; Liu *et al.*, 2024; Duan *et al.*, 2024; Paglieri *et al.*, 2025; Ruoss *et al.*, 2025), so when one model beats another they cannot tell whether size or training recipe made the difference.

There is a second, less discussed variable. A language model is a distribution over text, not a policy, and something has to turn one into the other. The usual choice is to generate text and parse an action out of it, which mixes formatting failures with bad decisions (Huang *et al.*, 2022; Liu *et al.*, 2024). One can instead score every legal action by its likelihood (Carta *et al.*, 2023; Tan *et al.*, 2024), read the decision off the hidden states with a probe (Belinkov, 2022; Orgad *et al.*, 2025), or fine-tune on expert actions (Chu *et al.*, 2025). These methods use the weights in different ways, and in multiple-choice question answering they disagree with each other more often than one would hope (Robinson and Wingate, 2023; Wang *et al.*, 2024). Papers on them usually study one method at one or two model sizes. A small model that fails when prompted might succeed when scored or probed; we do not know how often that happens, or whether it depends on the kind of decision.

The closest existing studies each cover part of this space. Jiwatode *et al.* (2026) vary Qwen3 model size, thinking mode and planning horizon on three spatial-navigation games, with one model family and prompting as the only conversion. Schmied *et al.* (2026) study models from 2B to 27B on bandits, contextual bandits and tic-tac-toe, identify greediness, frequency bias and a knowing-doing gap, and show that RL fine-tuning on the model’s own reasoning increases exploration and narrows the knowing-doing gap; their comparison is between prompting and RL fine-tuning, not across a set of conversion methods on a size ladder. Monea *et al.* (2025) span 500M to 70B parameters, but for in-context bandit learning on classification tasks, and Nie *et al.* (2025) relate bandit regret to model size and test inference-time and distillation interventions. None of these crosses same-recipe model ladders with several conversion methods and several kinds of decision problem, normalizes against an oracle, or fixes its hypotheses before collecting data. Some of our hypotheses already have partial answers in this work, and Section 2 says where. For those, what we add is a controlled, within-family and pre-registered measurement, not a new direction of effect.

Our contributions are the following.

Research questions. We ask five questions, the first four pre-registered and the fifth exploratory. RQ1: within a model family trained with one recipe, how does decision quality scale with parameter count, and does the slope depend on the structure of the task? RQ2: does the conversion method (free reply, formatted generation, likelihood scoring, a probe on hidden states, LoRA behavior cloning) change that dependence on scale? RQ3: which other factors move decision quality, and by how much: instruction tuning, quantization, reasoning budget, observation format, prompt encoding, and the data budget of adaptation? RQ4: what is the frontier of decision quality against inference cost, measured as latency, tokens and parameters? RQ5: how do small open-weight models converted with transparent methods compare with the purpose-built decision models that appeared in late 2026 and run on the same machine, on the same tasks, seeds and metrics, including the calibration of the probabilities they return?

- **A benchmark.** Seven procedurally generated decision tasks, each with an exact oracle, covering exploration (a multi-armed bandit), in-context rule learning (a contextual bandit), one-shot tabular decisions under covariate shift and under a reversed effect (loan approval), deterministic planning (gridworld), adversarial play (tic-tac-toe), risk (blackjack) and the application of a written policy to a customer’s request (support triage). A seed fixes an instance and all of its randomness, so policies are compared on identical potential outcomes, and returns are mapped onto a scale on which the random policy scores 0 and the oracle 1 (Section 3).
- **A harness.** Six methods behind one interface: the assistant’s unconverted free reply as the reference point, free-form generation, exact likelihood scoring of legal actions, a logistic probe on hidden states, LoRA behavior cloning, and the same probe applied to raw task features as a control that uses no language model. The harness runs local Hugging Face models on CUDA GPUs, on Apple silicon through Metal or on the CPU, and models behind any OpenAI-compatible server; it logs every decision with the raw model output and regenerates every result table and figure in this paper from those logs.

- **A pre-registered study.** Four within-family ladders from 135M to 8B parameters (SmolLM2, Qwen2.5, Qwen3 and Gemma 3), with the base and the instruct checkpoint at every size, 28 checkpoints in all, each run with every conversion method the hardware allows; LoRA covers the 24 checkpoints of at most 4B. Pythia and OLMo 2 serve as optional, fully open controls, five ablation blocks vary one factor at a time, seeds are paired throughout, and the falsification criteria of seven hypotheses are frozen at a git tag before the main sweep (Section 4).
- **A study that fits on one laptop.** Every number in the paper comes from one Apple M2 Max with 32 GB of memory. The models run with 16-bit weights through PyTorch’s Metal backend, except in the quantization ablation, where llama.cpp serves GGUF files on the same machine. The configurations, the order of the runs and the time each takes are published, so the whole study can be repeated by anyone with similar hardware. What the laptop cannot run, from the rungs above 8B to long reasoning budgets and the hosted decision models, is set out in Section 8 as concrete next steps for someone with more hardware.
- **Findings.** [result pending: [One sentence per hypothesis, for example: the slope under generation is b per tenfold increase in parameters and is steeper on planning tasks (H1); with LoRA, the smallest rung of a ladder matches the largest, prompted zero-shot and 12 to 15 times larger, on k of seven tasks in distribution but not under shift (H3); scoring closes $Z\%$ of the small-to-large gap (H4); Q4_K_M weights cost q at 1.5B (H5); no prompted model reaches UCB1 on the bandit (H6); a 256-token reasoning budget helps planning only from m B upward (H7)]] (Sections 5 and 6).

Section 2 reviews related work and marks where it already answers parts of our hypotheses. Section 3 defines the tasks, the score and the conversion methods, and Section 4 the models, factors and statistics. Section 5 tests each hypothesis against its pre-registered criterion. Section 6 examines errors, cost and the language-model-free control, Section 7 sets out what the design cannot tell us, and Section 8 describes what a single laptop could not run and how to continue with more hardware. The appendices give every prompt verbatim, the task generators, the full pre-registration, the compute budget, a reproducibility checklist and the verified model identifiers.

2 Related work

Language models as agents and decision-makers. SmartPlay (Wu *et al.*, 2024) probes nine capabilities with six games, one of them a two-armed bandit, and finds that models differ by capability rather than by a single agentic skill. BALROG (Paglieri *et al.*, 2025) grades difficulty from BabyAI to NetHack; models make progress on the easy environments and fail on the hardest. LMAct (Ruoss *et al.*, 2025) shows that even 512 expert episodes in context rarely bring frontier models to expert level on tasks that include tic-tac-toe and grid navigation. AgentBench (Liu *et al.*, 2024) evaluates 29 API and open models in eight environments and names weak long-term reasoning, decision-making and instruction following as the main obstacles, and GTBench (Duan *et al.*, 2024) finds that LLMs fail in complete-information deterministic games such as tic-tac-toe. Huang *et al.* (2022) showed that plans generated by language models are often not admissible actions, the problem that likelihood scoring avoids by construction. These benchmarks establish that difficulty depends on task structure, the premise of the task-dependent slope in H1, but their model sets are not controlled ladders and each fixes one conversion method. The design nearest to ours is that of Jiwatode *et al.* (2026), who cross Qwen3 model size with thinking mode and planning horizon on spatial games; we extend it to several families, five conversion methods and seven decision structures.

Exploration and bandits. Krishnamurthy *et al.* (2024) found that, across many prompt designs, only GPT-4 with chain-of-thought and an externally summarized history explored a multi-armed bandit reliably. EVOLvE (Nie *et al.*, 2025) relates regret to model size and shows that algorithm-guided inference and algorithm distillation let smaller models explore better than larger ones without such support. Schmied *et al.* (2026) trace the failures of 2B to 27B models to greediness, with up to 55% of the action space left unexplored, to a frequency bias that fades with scale, and to a knowing-doing gap: rationales were correct about 87% of the time, yet the chosen action often did not follow from them. Harris and Slivkins (2026) find that LLMs often fail even to exploit, falling short of simple linear regression, and Monea *et al.* (2025) show that models from 500M to 70B parameters can learn from reward in context. Algorithm distillation

bears on how such policies should be trained: cloning the learning histories of a reinforcement-learning algorithm, not expert actions, is what teaches in-context reinforcement learning ([Laskin et al., 2023](#)). We use UCB1 ([Auer, Cesa-Bianchi and Fischer, 2002](#)) as a realizable reference and standard regret definitions ([Lattimore and Szepesvári, 2020](#)). H6, that exploration is poor without scaffolding, is therefore largely established. What we add is its dependence on scale within families, its behavior under conversion methods other than prompting, and a separation of exploration errors from exploitation errors.

Scale, emergence and small models. Pre-training loss falls predictably with parameters and training tokens ([Hoffmann et al., 2022](#)), which motivates the log-linear form of H1. The same work complicates any reading of such a slope, because the small rungs of modern ladders are trained far beyond the compute-optimal number of tokens; SmoLLM2-1.7B, for example, saw about 11T tokens ([Ben Allal et al., 2025](#)). Parameter count therefore partly stands in for tokens per parameter. Emergent abilities ([Wei, Tay, et al., 2022](#)) are the main alternative to a smooth slope, and Schaeffer, Miranda and Koyejo ([2023](#)) attribute many of them to discontinuous metrics, one reason we fit slopes to continuous normalized returns rather than to success rates. Few-shot ability improves steadily with scale within one recipe ([Brown et al., 2020](#)), and the length of task a model can execute keeps growing with size after single-step accuracy has saturated ([Sinha et al., 2026](#)). The case for small models rests on extra computation and adaptation. In FLOPs-matched comparisons a small model given more test-time compute can beat one 14 times larger on problems it already solves some of the time ([Snell et al., 2025](#)), and algorithm distillation lets small models explore bandits better than larger unaided ones ([Nie et al., 2025](#)), which partly answers the in-distribution half of H3 for that task family.

Converting a language model into a policy. Scoring answer strings by probability is distorted by surface-form competition between paraphrases ([Holtzman et al., 2021](#)). Scoring an option symbol works only for models that bind symbols to options well ([Robinson and Wingate, 2023](#)), option identifiers carry a token-level prior ([Zheng et al., 2024](#)), and few-shot predictions lean toward majority, recent and common labels ([Zhao et al., 2021](#)). For instruction-tuned models, first-token probabilities often disagree with the answer the model writes ([Wang et al., 2024](#)). Meaning-preserving changes of prompt format move few-shot accuracy by up to 76 points ([Sclar et al., 2024](#)), and the serialization of tabular inputs matters for LLM classifiers ([Hegselmann et al., 2023](#)). As a policy, likelihood scoring of legal actions underlies GLAM ([Carta et al., 2023](#)) and TWOSOME ([Tan et al., 2024](#)), and the latter shows that long action strings are penalized unless scores are normalized for length. This work motivates H4, but it was measured on question answering, not on sequential decisions across a size ladder.

A linear probe on frozen hidden states shows what is decodable, which is not the same as what the model uses, and needs control tasks to be interpreted ([Belinkov, 2022](#)). Intermediate layers are often better features than the last ([Skean et al., 2025](#)), and models often encode the correct answer while generating a wrong one ([Orgad et al., 2025](#)). That gap is how a probe could beat generation on our tasks, and it is why we pair the probe with a control trained on raw task features.

Behavior cloning ([Pomerleau, 1988](#)) compounds its errors once the learner leaves the expert’s state distribution ([Ross, Gordon and Bagnell, 2011](#)), and LoRA ([Hu et al., 2022](#)) and QLoRA ([Dettmers et al., 2023](#)) make it affordable across a wide range of model sizes. Whether a cloned policy generalizes is contested. Supervised fine-tuning memorizes the rules it was trained on while outcome-reward RL generalizes ([Chu et al., 2025](#)), which supports the out-of-distribution half of H3; frozen language models with RL-trained adapters ([Szot et al., 2024](#)) and LoRA fine-tuning on human choice data ([Binz et al., 2025](#)) generalize to some shifts, which cuts against it. LIMA ([Zhou et al., 2023](#)) and URIAL ([B. Y. Lin et al., 2024](#)) suggest that alignment mostly changes style and format. That predicts the second clause of H2, that base and instruct checkpoints converge under probing and fine-tuning, but it also threatens the first, since a base model with a good few-shot prompt may close the gap under prompting as well. DPO ([Rafailov et al., 2023](#)), GRPO ([Shao et al., 2024](#)) and self-play on text games ([Liu et al., 2026](#)) are conversions that DEAD EYE v1 leaves out.

Reasoning and output format. Chain-of-thought helps large models, while small ones produce fluent but illogical chains ([Wei, Wang, et al., 2022](#)). More reasoning can also hurt. Deliberation lowers accuracy on tasks where it hurts people too, such as implicit statistical learning ([Liu, Geng, Wu, et al., 2025](#)); on controllable puzzles, thinking helps only at intermediate complexity ([Shojaee et al., 2025](#)); longer reasoning can reduce accuracy ([Gema et al., 2025](#)); and models of about 3B parameters or fewer do not reliably benefit from long chains ([Li et al., 2025](#)). Strict output formats can suppress reasoning ([Tam et al., 2024](#)), so our chain-of-thought prompt lets the model reason freely before a delimited

action line. Every clause of H7 finds support somewhere in this work, but on different tasks and models. Switching thinking on and off inside the same weights ([Yang et al., 2025](#)) permits a cleaner test, and latency-normalized efficiency has not been compared across sizes and task structures.

Quantization. At a fixed total bit budget, 4-bit weights are close to optimal on zero-shot language tasks ([Dettmers and Zettlemoyer, 2023](#)), and GPTQ ([Frantar et al., 2023](#)) and AWQ ([J. Lin et al., 2024](#)) quantize large models with little loss. Precision-aware scaling laws predict that the penalty for post-training quantization grows with the amount of pre-training data ([Kumar et al., 2025](#)); small modern models are the most over-trained, so this is the mechanism behind H5. These studies measure loss, perplexity or accuracy on language benchmarks, so for H5 there is a mechanism but no direct evidence on multi-step decisions.

Evaluation methodology. We follow Agarwal *et al.* ([2021](#)) in reporting interquartile means and bootstrap confidence intervals, and Henderson *et al.* ([2018](#)) in fixing seeds and hyperparameter budgets across all rungs. Because results for language models depend heavily on prompt format and on the choice between scoring and generation ([Sclar et al., 2024](#); [Wang et al., 2024](#)), we report both conversions side by side and release every decision.

Cognitive and behavioral studies of decisions. Studies in the style of experimental psychology ask how models decide, not only how well. GPT-3 outperformed people on a two-armed bandit but showed no directed exploration ([Binz and Schulz, 2023](#)). Even GPT-4 falls short of human-like rationality when it must form and keep beliefs ([Fan et al., 2024](#)), language models show human-like risk and loss aversion ([Jia et al., 2024](#)), and they assume that people are more rational than people are ([Liu, Geng, Peterson, et al., 2025](#)). Our oracles are normative rather than human, but these findings suggest examining the direction of errors, for instance whether a model stands too early in blackjack, and not only their rate (Section 6).

Decision models. While this study was being prepared, “decision models” became a product category. TypeSafe AI’s Jev, announced in September 2026, takes a state and typed questions and returns a chosen option with a probability for every allowed answer and a confidence, never prose; the company describes it as a “System One” model trained on synthetic data with a method it calls reinforcement learning for calibrated decisions, and keeps the weights and the architecture closed ([TypeSafe AI, 2026b, 2026c, 2026a](#); [TechCrunch, 2026a](#)). Open-weight relatives followed within weeks. Kev attaches a rank-16 LoRA adapter and a small pointer head to Qwen3.5 and Qwen3.8 bases at 0.8B to 27B parameters, ships under Apache-2.0 with a server that speaks the same protocol, and reports accuracy close to Jev’s on held-out classification sets while noting that Jev’s training data is unknown ([Palmer, 2026](#)). Cloudflare’s Clef and Clef-flash freeze Qwen3.8-27B and Qwen3.5-9B and train a routing head with low-rank adapters, are served on Workers AI with vendor-reported median latencies of about 209 and 39 milliseconds against about 524 for Jev, and are likewise Jev-compatible ([Cloudflare, 2026](#)); Perplexity’s pplx-decider fine-tunes Qwen3.8-27B and reports a small lead over Jev on an eleven-task panel ([Perplexity AI, 2026](#)); Amazon released a 2B decider of its own ([TechCrunch, 2026b](#); [The New Stack, 2026](#)). Two smaller systems take different routes. Laya, from Convai Innovations, puts a decision head on encoder models rather than on a generative base, ModernBERT-large at 421M parameters and mmBERT-base at 322M, under Apache-2.0, and reports a few tens of milliseconds per question on a Tesla T4 ([Convai Innovations, 2026](#)). SemIf, formerly OpenJev, trains nothing at all: it reads the logits of the option labels from a frozen open checkpoint such as Qwen3.5-4B in one forward pass, which is precisely the letter-format likelihood scoring we study ([Lee, 2026](#)). In the terms of this paper these systems are the probe and LoRA conversions productized: an open base, frozen or lightly adapted, plus a head that reads probabilities over the allowed answers, exposed through an interface that is exactly our likelihood scoring. What they have not been evaluated on is decision quality against a known optimal policy; their published numbers are label accuracies on classification panels, mostly measured by the vendors themselves. DEAD EYE treats them as one more backend. Those that run on a laptop, Kev at 0.8B and 4B, Laya and SemIf, are placed on the same tasks, seeds and normalized scale as the conversions we study (Section 4.6); the hosted systems and the larger checkpoints are left to future work (Section 8).

3 The DEAD EYE benchmark

A DEAD EYE task is a generator of small decision problems, each fixed completely by a seed and each paired with an oracle. This section defines the tasks and the score, then the six ways we turn a language model into a policy, and finally what the harness records.

3.1 Decision tasks

A task $\mathcal{T}$ maps a seed $\sigma \in \mathbb{N}$ to an episodic decision problem. Its generator $G_{\mathcal{T}}$ seeds a pseudo-random number generator from σ and draws two things: hidden instance parameters ξ_{σ} , such as the arm means of a bandit, and all the exogenous randomness ω_{σ} the episode can ever consume, such as a table holding the reward of every arm in every round. Given these, an episode is a deterministic finite-horizon problem

$$M_{\sigma} = (\mathcal{S}, \mathcal{A}, \mathcal{A}(\cdot), s_0, f, r, H), \quad s_{t+1} = f(s_t, a_t; \xi_{\sigma}, \omega_{\sigma}), \quad r_t = r(s_t, a_t; \xi_{\sigma}, \omega_{\sigma}),$$

where $\mathcal{A}$ is the set of action labels, $\mathcal{A}(s) \subseteq \mathcal{A}$ the actions that are legal in state s , and H a cap on the number of decisions. The episode ends in a terminal state or after H decisions, and its return is $R(\pi, \sigma) = \sum_t r_t$. Before each decision the policy receives an observation $o_t = (x_t, \mathcal{A}(s_t), \phi(s_t))$: a text rendering x_t of the state, the legal actions, and a numeric feature vector $\phi(s_t)$ that only the language-model-free control and the classical baselines use. Language-model policies also receive a fixed description of the rules. Every task has an oracle $\pi^*(s; \xi_{\sigma})$ that returns an optimal action, chosen by a fixed rule when several actions are optimal.

Because ω_{σ} is drawn before the first decision, two policies run on the same seed face identical potential outcomes: if both pull arm 3 in round 7 of seed 12, both receive the same reward. This is the method of common random numbers (Law, 2015), and it makes every comparison in this paper a paired comparison over seeds.

The oracles differ in what they know. On gridworld, tic-tac-toe and blackjack the oracle is a function of what the text shows, so a policy that reads the text perfectly can match it. On loan approval the oracle uses a default model that is the same in every instance; the text shows every input of that model but not its coefficients, which a policy has to bring as prior knowledge or learn from training seeds. On the two bandits the oracle knows the hidden arm means or weights. It is clairvoyant, no policy that learns from feedback can match it in expectation, and a normalized score of 1 is an upper bound rather than a target there; UCB1 provides a realizable reference on the bandit (Section 3.5).

3.2 Normalized score

Raw returns are not comparable across tasks, so we place them on a common scale anchored by two reference policies run on the same seeds:

$$s(\pi, \sigma) = \frac{R(\pi, \sigma) - \bar{R}_{\text{rand}}}{\bar{R}_{\text{orc}} - \bar{R}_{\text{rand}}}, \quad \bar{R}_{\text{rand}} = \frac{1}{|\Sigma|} \sum_{\sigma \in \Sigma} R(\pi_{\text{rand}}, \sigma), \quad \bar{R}_{\text{orc}} = \frac{1}{|\Sigma|} \sum_{\sigma \in \Sigma} R(\pi^*, \sigma),$$

where Σ is the set of evaluation seeds and π_{rand} chooses a legal action uniformly at random. The anchors are means over Σ , so the random and the oracle policy average exactly 0 and 1, while a single episode can score below 0 or above 1. Every task configuration, for example the ten-armed bandit or the sign-flipped loan task, gets its own anchors from its own run. For each cell we report the mean and the interquartile mean of s over episodes with a bootstrap confidence interval, and we summarize a model and method by the unweighted mean over the tasks it was run on, all seven in the main sweep. Because s is an affine function of return, it preserves every comparison that return supports; regret, success and win rates are reported alongside. The scaling has a price: when the two anchors lie close together, as in blackjack, the per-episode spread of s is large, and statistical power suffers (Section 4.5).

3.3 Tasks

Table 1 summarizes the seven tasks; Appendix B lists every parameter and distribution, and Appendix A shows a prompt for each.

The seven tasks with the parameters of the main sweep. “Decisions” is the number of decisions per episode, which varies with the policy on gridworld, tic-tac-toe and blackjack.

Task	Decision structure	Oracle	Headline metric	Decisions
bandit	exploration and exploitation without state; 5 arms	best true arm (clairvoyant)	pseudo-regret	50
contextual_bandit	learning an abstract rule in context; 3 options	best option for the context (clairvoyant)	regret	30
loan	one-shot tabular decisions with semantic features; shift variants	approve iff expected value is positive	profit, regret	20
gridworld	deterministic multi-step planning on a 6×6 grid	first step of a shortest path	success, excess steps	≤ 24
tictactoe	adversarial, perfect information; random opponent	expectimax against the random opponent	win rate	≤ 5
blackjack	risk under uncertainty; hit or stand; 20 hands per episode	hit/stand basic strategy	mean return per hand	≥ 20
support	applying a written six-rule policy to a customer’s message; 6 actions	the policy’s rules	correct-decision rate, wasted payout	20

Multi-armed bandit (bandit). The agent plays K Bernoulli arms for T rounds ($K = 5, T = 50$ in the main sweep; $K = 10, T = 100$ in a harder variant). Following Krishnamurthy *et al.* (2024), one arm a^* , chosen uniformly, has mean $\mu_{a^*} = 0.5 + \Delta/2$ and every other arm has $\mu_a = 0.5 - \Delta/2$, with gap $\Delta = 0.2$. The generator pre-draws $Y_{t,a} \sim \text{Bernoulli}(\mu_a)$ for every round and arm, and pulling arm a_t in round t pays Y_{t,a_t} . The observation gives the round number and, by default, a summary with the number of pulls and the average reward of each arm; the raw format lists the full history of (arm, reward) pairs instead. The oracle pulls a^* every round. The headline metric is pseudo-regret,

$$\text{Reg}_T = \sum_{t=1}^T (\mu_{a^*} - \mu_{a_t}),$$

and we also log the fraction of suboptimal pulls and the share of the last ten pulls that went to a^* . The feature vector holds each arm’s share of past pulls and its empirical mean, set to 0.5 for arms not yet pulled.

Contextual bandit (contextual_bandit). Each of $T = 30$ rounds shows a context $x_t \in \mathbb{R}^d$ with $d = 4$ and offers $A = 3$ options. Hidden weights $w_a \sim \mathcal{N}(0, \sigma_w^2 I_d)$ with $\sigma_w = 2$ are drawn for each episode, contexts are drawn from $\mathcal{N}(0, I_d)$ and rounded to two decimals, and option a succeeds with probability

$$p_{t,a} = \text{sigmoid}(w_a^\top x_t / \sqrt{d}),$$

with outcomes again pre-drawn for every round and option. The observation shows the current context and the last ten rounds as (context, choice, reward) triples. The features have abstract names (x1 to x4) and the weights change every episode, so prior knowledge cannot help: the only route above chance is to infer the rule from the history in the prompt. The oracle chooses $\arg \max_a p_{t,a}$, regret is $\sum_t (\max_a p_{t,a} - p_{t,a_t})$, and the feature vector is the current context alone.

Loan approval (loan). An episode presents $N = 20$ independent applicants, each with an age, an annual income I , a debt-to-income ratio D , a credit score C , years in current employment E , a requested amount L and a loan purpose. Incomes and amounts are log-normal, the debt ratio follows a Beta(2, 5) distribution and credit scores are normal around 680. The probability that an applicant defaults is

$$p = \text{sigmoid}\left(-1.2 + 2c \frac{C-680}{70} + 2.5 (D - 0.28) + 1.2 \ln(1 + L/I) - 0.08 \min(E, 15) + \rho_{\text{purpose}}\right),$$

with $c = -1$, so that a higher credit score lowers the risk, and a purpose offset ρ between -0.3 (home improvement) and $+0.6$ (small business). Age is displayed but does not enter the model. Approving earns rL with $r = 0.12$ if the loan is repaid and loses λL with $\lambda = 0.6$ on default; denying earns nothing; defaults are pre-drawn. The expected value of an approval is

$$\mathbb{E}[\text{approve}] = (1 - p)rL - p\lambda L,$$

so the oracle approves exactly when $p < r/(r + \lambda) = 1/6$. Each prompt states the gain and the loss in dollars, which reduces the decision to estimating p . The headline metrics are profit and the regret

$\sum_i (\max(\mathbb{E}_i[\text{approve}], 0) - \mathbb{E}_i[\text{chosen}])$, where the chosen action’s expected value is zero for a denial. Two variants test distribution shift with the same generator. Under covariate shift the median income falls from 55k to 35k dollars and the mean credit score from 680 to 600, with the default model unchanged. Under the sign flip $c = +1$: a higher credit score now raises the risk, and a model’s prior knowledge points the wrong way. The feature vector holds the six numeric fields, standardized, and a one-hot purpose.

Gridworld navigation (gridworld). The agent walks on an $n \times n$ grid toward a goal ($n = 6$ in the main sweep; $n = 9$ with 16 walls in a larger variant). Start, goal and n wall cells are distinct cells taken from a random permutation, and the instance is redrawn until the goal is reachable at a shortest-path distance $d^* \geq n$. The legal actions are the moves among up, down, left and right that stay on the grid and avoid walls. Each move costs 0.02 and reaching the goal pays 1, so a successful episode of t moves returns $1 - 0.02t$ and the oracle returns $1 - 0.02d^*$; the episode stops at the goal or after $H = 4n$ moves. The observation is an ASCII map with the coordinates of the agent and the goal. The oracle takes the first step of a breadth-first-search shortest path, breaking ties in the fixed order up, down, left, right. We report success and excess steps, $t - d^*$ on success and $H - d^*$ otherwise. The feature vector holds the normalized coordinates of agent and goal and the wall mask.

Tic-tac-toe (tictactoe). The agent plays a full game against a fixed opponent that moves uniformly at random in the main sweep and by minimax in a variant. It plays X, and moves first, on even seeds, and O on odd seeds. The board is shown with empty cells numbered 1 to 9, and the legal actions are the empty cells. A win pays $+1$, a draw 0 and a loss -1 . Against the random opponent the oracle is expectimax: it maximizes the expected outcome against uniformly random replies and, among moves of equal expected value, takes an immediate win when one is available and otherwise the lowest-numbered cell. Against the minimax opponent the oracle is minimax with depth-aware values, a terminal win being worth $1 + e$ and a loss $-(1 + e)$ with e the number of empty cells left, so that it prefers fast wins and slow losses. That opponent breaks ties between equally good moves at random, with a generator seeded by the episode seed, so that a deterministic policy meets many different games, and no policy can beat it. The headline metric is the win rate, with draws and losses logged; the feature vector encodes the board from the agent’s point of view.

Blackjack (blackjack). An episode is a sequence of H hands against a dealer ($H = 1$ by default and $H = 20$ in the sweep, where the observation also shows the hand number and the running total); each hand follows the Gymnasium convention ([Towers et al., 2025](#)): an infinite deck in which each card is drawn uniformly from $\{1, \dots, 9, 10, 10, 10, 10\}$ (an ace, two to nine, and four ten-valued ranks), a dealer who draws to 17 and stands on soft 17, and a payout of 1.5 for a winning two-card 21. One ace counts as 11 whenever that does not take the hand over 21. The only actions are hit and stand; there is no doubling, splitting, surrender or insurance. Each seed pre-draws two card streams, one for the player and one for the dealer, so a policy’s hits never change the dealer’s cards and every policy faces the same dealer hand. The oracle is the hit/stand basic-strategy table ([Baldwin et al., 1956](#)): with a hard total it stands on 17 or more, on 13 to 16 against a dealer card from 2 to 6, and on 12 against 4 to 6; with a soft total it stands on 19 or more and on soft 18 against 2 to 8. For these rules the table coincides with the optimal hit/stand policy, which we checked by dynamic programming over the player’s total, the soft-hand flag and the dealer’s card ([Sutton and Barto, 2018](#)). It is optimal only within the hit/stand game, since the task offers no doubling or splitting. The episode return is the sum of the hand outcomes, so bundling H hands divides the standard deviation of the normalized score by about $\sqrt{H}$ without changing the decision problem. The headline metric is the mean return per hand, with win, loss and bust rates logged; the features are the hand total, a soft-hand flag and the dealer’s up card.

Customer-support triage (support). The agent resolves n support tickets in a row ($n = 20$ in the sweep). Each ticket is a customer’s message, such as “I received the package damaged. I need a refund today.”, together with the order facts an agent would see: the item and its value, when it was delivered or how late it is, whether photo evidence is attached, how many claims the customer has made before, the customer’s tier and whether a replacement is in stock. The task description states the company’s six-rule policy, which covers repeat claimers, billing errors, changed minds, late deliveries, missing deliveries and damaged or wrong items, and the six actions are refund, replace, partial refund, request evidence, escalate and deny. The oracle applies the rules literally, so this task measures whether a model can follow a written policy on a concrete case rather than whether it has a good prior. A correct decision earns $+1$ and a wrong one -1 ; an unwarranted payout costs its value in hundreds of dollars on top, and refusing a claim the policy would have paid costs a further 1 for the lost customer. The headline metric is the share of decisions that match the policy, with the wasted payout logged; the features are the one-hot issue and request plus the scaled order fields.

3.4 Conversion methods

A conversion method turns a language model, or in one case no model at all, into a policy (Table 2). All six share one prompt skeleton, so that two conditions differ only in the factor under study. The first of them, the free reply, is not a conversion at all: it is the assistant as shipped, answering the situation in its own words, and it is the reference against which the cost and the reliability of every real conversion are measured.

The six conversion methods. Only the two generation methods can produce an illegal action.

Method	What it uses	Training data	Backends
prompt_free	the unconverted assistant’s free reply, with an action extracted when the text contains one	none	hf, openai
prompt_generate	generated text, parsed into an action (zero-shot, few-shot or chain-of-thought)	none	hf, openai
prompt_score	log-likelihood of each legal label as a continuation of the prompt	none	hf (exact), openai (first token), decision (a decision model’s probabilities)
probe	last-token hidden state at one layer, with a logistic-regression head	oracle-labeled states from training seeds	hf
lora_sft	LoRA adapters trained by behavior cloning, then exact scoring	the same states	hf
feature_probe	no language model: the same head on the task’s numeric features	the same states	any

Prompt skeleton. The system message is the task description. The user message holds the observation text, a line `Legal actions:` that lists the legal labels, and an answer instruction. In the *name* format the labels are the action names themselves (`arm_3`, `left`, 7). In the *letter* format the legal actions receive the letters A, B, C and so on, in order, and one line per letter states its action. The instruction asks for a single line `Action: <label>` in generation mode, for at most three short sentences of reasoning followed by that line in chain-of-thought mode, and for the label alone in scoring mode. Few-shot demonstrations are complete user and assistant turns placed before the final user message; each shows a state from a training seed, reached after zero to four oracle steps, together with the oracle’s action. Instruction-tuned checkpoints receive the messages through their chat template. Base checkpoints receive a plain rendering in which user and assistant turns are introduced by lines reading `Input:` and `Output:`, even if their tokenizer ships a chat template. The model sees its own past decisions only where the observation itself shows them, as in the bandit summaries and the contextual-bandit history. Appendix A reproduces every variant verbatim.

Free reply (prompt_free). The user message ends with “What do you do? Answer in your own words.” instead of a format instruction, and the model decodes up to 96 new tokens. The parser then looks for an action in the reply exactly as it does for generation; a reply that names no legal action, or several without committing to one, counts as illegal. This

is the “before” condition: it records how long an unconverted assistant takes, how many tokens it spends, and how often its answer contains a usable decision at all.

Free-form generation (prompt_generate). The model decodes greedily up to 16 new tokens, or up to 256 with chain-of-thought and with thinking mode in the reasoning ablation; one ablation samples at temperature 0.7 instead. Longer thinking budgets are left to future work (Section 8). Greedy decoding overrides any repetition penalty that a checkpoint ships with, so base and instruct checkpoints decode the same way. A parser then maps the text to a label. It removes <think> blocks, including an unterminated block left when thinking exhausted the budget, and reads the last line of the form Action: ..., JSON-style lines included, whose content matches a legal label once quotes, brackets and punctuation are stripped. Single-character labels (letters and tic-tac-toe cells) must match exactly, apart from a leading word such as *option* or *cell*; longer labels may also match as a prefix, as a whole word, or by close spelling (a similarity ratio of at least 0.8). Without such a line the parser takes the last line that matches a label under the same rules and, for multi-character labels only, finally the last label mentioned anywhere in the text. Anything else is a format failure.

Likelihood scoring (prompt_score). Scoring never asks the model to write anything. For each legal label c , tokenized on its own as $(c_1, \dots, c_m)$, the score is its log-probability as a continuation of the rendered prompt,

$$\ell(c) = \sum_{j=1}^m \log p_{\theta}(c_j \mid \text{prompt}, c_{<j}), \quad \ell_{\text{norm}}(c) = \ell(c)/m,$$

and the policy takes $\arg \max_c \ell(c)$, or $\arg \max_c \ell_{\text{norm}}(c)$ in the length-normalization ablation, with ties going to the first label. All labels are scored in one batched forward pass; this is the cloze-style scoring used for multiple-choice evaluation (Holtzman et al., 2021; Robinson and Wingate, 2023). When the tokens of one label are a strict prefix of another’s, as for `arm_1` and `arm_10` under a tokenizer that splits digits, every label is scored with an end-of-sequence token appended; otherwise the longer label could never win. The policy cannot produce an illegal action. Models behind an OpenAI-compatible API expose only the top log-probabilities of the first generated token, so for them each label is scored by its first token. That is exact only when the labels begin with distinct tokens, which is why we run such models with letter labels and report them separately. In this study the only such models are the GGUF quantizations of the quantization ablation, which llama.cpp serves on the same laptop (Section 4.2).

Hidden-state probe (probe). The probe reads a decision from the model’s representation of the prompt without generating anything. Let $h_{\ell}(x)$ be the hidden state of the last prompt token after block ℓ of a model with B blocks, counting the embedding output as $\ell = 0$. We use the middle layer, $\ell = \lfloor B/2 \rfloor$, and in an ablation the last layer, $\ell = B$. A multinomial logistic regression with an L_2 penalty (inverse strength $C = 1$), fitted with scikit-learn (Pedregosa et al., 2011), is trained on the standardized h_{ℓ} to predict the oracle’s action. The training states come from the first N_{train} training seeds, 100 in the main sweep and 10, 40 or 100 in the data-budget ablation. A behavior policy that takes the oracle’s action with probability 0.7 and a uniformly random legal action otherwise visits them, so that states off the oracle’s path are covered too, and every visited state is labeled with the oracle’s action. Collection stops at 50000 states by default, a cap that no probe configuration reaches, and at 3000 states in the LoRA configurations; the numbers of training episodes and states actually used are logged. For the two shifted loan variants the training episodes come from the unshifted task, so adapted policies are evaluated out of distribution there; few-shot demonstrations follow the same rule. At decision time the probe takes the legal action with the highest predicted probability. It embeds the generation-mode prompt.

LoRA behavior cloning (lora_sft). LoRA adapters (Hu et al., 2022) of rank 16 with scale $\alpha = 32$ and dropout 0.05 are attached to every linear layer and trained for two epochs, with AdamW, a learning rate of 2×10^{-4} , batch size 8, no weight decay and gradient norms clipped at 1. The training states are collected as for the probe, from the first 40 training seeds in the main LoRA configuration and at most 3000 of them; the data-budget ablation uses 10 and 40 seeds, and 100 at its two smallest sizes. Each example is the scoring-mode prompt followed by the oracle’s label and an end-of-sequence token, and the cross-entropy loss covers only those target tokens. After training the adapters are merged into the 16-bit weights, and the policy acts by exact scoring with the same prompt. Every LoRA cell starts from a fresh copy of the pre-trained weights. Training time grows with model size, and on the laptop LoRA runs only on checkpoints of at

most 4B (Section 4.4). QLoRA ([Dettmers et al., 2023](#)), which trains adapters on 4-bit weights and would carry the method to the larger rungs, needs CUDA and is left to future work.

Feature probe (feature_probe). The control uses no language model. It is the same logistic regression, trained on the same states, with the task’s feature vector $\phi(s)$ in place of h_ℓ . What it controls for differs by task, and Section 6.3 discusses how to read it.

Illegal actions. Only the two generation methods, the free reply and free-form generation, can fail to yield a legal action. When the output cannot be mapped to a legal label, the runner substitutes a legal action drawn uniformly at random, from a generator seeded by the episode seed, and marks the decision as illegal. The substituted action counts toward the return but never toward oracle agreement. The illegal-action rate, the share of decisions marked illegal, is a secondary outcome. The harness can instead take the first legal action or end the episode; we use random substitution throughout, so that a format failure is scored like an uninformed decision rather than a forfeit.

3.5 Baselines

Three policies need no language model. The random policy draws a legal action uniformly, from one generator per cell. The oracle is π^* . On the bandit we also run UCB1 ([Auer, Cesa-Bianchi and Fischer, 2002](#)), which pulls each arm once and then always the arm that maximizes

$$\hat{\mu}_a + \sqrt{2 \ln(t+1)/n_a},$$

where t is the number of completed rounds, n_a the number of pulls of arm a and $\hat{\mu}_a$ its mean reward so far. UCB1 is the textbook reference, not the strongest algorithm for a 50-round horizon. Its normalized score, [\[result pending: \[s of UCB1 on bandit with CI\]\]](#), marks what a non-clairvoyant learner with a regret guarantee achieves on this task.

3.6 Logging and provenance

A run crosses the task configurations, models and methods of a configuration file into cells, and evaluates each cell on every evaluation seed. Each cell writes three files to `results/<run>/<task>/<model>/<method>/`. The first, `summary.json`, holds the aggregates (mean and standard deviation of the return and of every task metric, illegal-action rate, oracle agreement, latency and prompt and completion tokens per decision) together with the full model and method specification, the model’s precision and parameter count, the seeds, the task parameters used for training, statistics from the training phase such as probe accuracy, LoRA loss before and after training and the number of training episodes used, timings and the harness version. The second, `episodes.jsonl`, has one record per episode: seed, return, task metrics, illegal-action count, oracle agreement, latency and token counts. The third, `steps.jsonl`, has one record per decision: legal actions, the oracle’s action, the parsed and the executed action, whether the decision was legal, the reward, latency, token counts, the raw model output truncated to 2000 characters, and method-specific extras such as the score of every label or the probe’s class probabilities. Each run directory also holds `run_manifest.json`, with the configuration, the list of cells, start and finish times, the platform string, the Python, PyTorch and Transformers versions, and whether CUDA was available, with the GPU’s name when it was. A cell whose summary already exists is skipped, so a run can be stopped and resumed, which is how the long configurations of this study run night after night on one laptop, or split across machines; `deadeye_report` rebuilds every result table and figure in this paper from these files.

4 Experimental setup

4.1 Model ladders

A scale effect means something only within a family trained with one recipe, so we evaluate ladders rather than a collection of popular models (Table 3). Every ladder stops at 8B parameters, the largest size whose 16-bit weights, about 16 GB, leave room for everything else in the laptop’s 32 GB (Section 4.4). Qwen2.5 ([Yang et al., 2024](#)) is the primary

ladder, with four dense sizes from 0.5B to 7B released together. Qwen3 (Yang *et al.*, 2025) adds four sizes from 0.6B to 8B whose post-trained checkpoints switch thinking on and off inside the same weights; we run them with thinking off everywhere except in the reasoning ablation. SmolLM2 (Ben Allal *et al.*, 2025) and Gemma 3 (Gemma Team, 2025) extend the low end to 135M and 270M parameters, with three sizes each. At every size we run both the base (pre-trained) and the instruct (post-trained) checkpoint, 28 checkpoints in all. Two fully open ladders serve as controls in a separate configuration that we run if time allows: Pythia (Biderman *et al.*, 2023), six base-only sizes from 70M to 2.8B trained on the same data in the same order, and OLMo 2 (Walsh *et al.*, 2025), with open data and base and instruct checkpoints at 1B and 7B. The larger rungs, Qwen2.5 from 14B to 72B, Qwen3 at 14B and 32B, Gemma 3 at 12B and 27B, Pythia at 6.9B and 12B and OLMo 2 at 13B and 32B, are left to future work (Section 8).

We leave out ladders that would put different kinds of model on one axis. The Llama 3 herd (Grattafiori, Dubey, *et al.*, 2024) covers 8B, 70B and 405B, and the smaller Llama sizes come from later releases; the DeepSeek-R1 distillations sit on base models from two families (Guo *et al.*, 2025); and mixture-of-experts models would force a choice between active and total parameters. Newer dense ladders, such as Olmo 3 (Team Olmo, Ettinger, *et al.*, 2025), can be added through the model catalog once their repository identifiers are verified.

Model ladders in this study, all run with 16-bit weights on one laptop. “Main” is the main sweep; “LoRA” and “free reply” are the two configurations that complete it, and “controls” the optional control configuration (Section 4.2). Rungs above 8B are discussed in Section 8, and Appendix F gives the repository identifiers.

Family	Sizes	Checkpoints	Configurations	License
Qwen2.5	0.5B, 1.5B, 3B, 7B	base, instruct	main; LoRA up to 3B; free reply (instruct, every size)	Apache-2.0; 3B under the Qwen Research License
Qwen3	0.6B, 1.7B, 4B, 8B	base, post-trained	main, thinking off; LoRA up to 4B; free reply (post-trained, every size); thinking on in the reasoning ablation	Apache-2.0
SmolLM2	135M, 360M, 1.7B	base, instruct	main; LoRA; free reply (instruct, 360M and 1.7B)	Apache-2.0
Gemma 3	270M, 1B, 4B	pre-trained, instruct	main; LoRA; free reply (instruct, 1B and 4B)	Gemma Terms of Use
Pythia	70M, 160M, 410M, 1B, 1.4B, 2.8B	base only (deduplicated)	controls	Apache-2.0
OLMo 2	1B, 7B	base, instruct	controls	Apache-2.0

Every checkpoint runs with 16-bit weights, bfloat16 where the Metal backend supports it and float16 otherwise, and every table marks the precision of each row. No rung is quantized, so the top of each ladder does not mix scale with quantization, as it would if the largest models were loaded with 4-bit weights; the price is that the ladders end at 8B. The parameter axis is the total parameter count from the model catalog. Embeddings take a large share of the smallest models, about a quarter of Qwen2.5-0.5B’s 0.49B parameters according to its model card, and more in models with large vocabularies, so we repeat the slope fits with non-embedding counts where they are published.

4.2 Factors and levels

Table 4 lists the factors we vary and the configuration that varies each. The main sweep, `configs/mac_main.yaml`, crosses the 28 checkpoints with four methods on the seven tasks: generation with a 16-token budget, likelihood scoring, the probe at the middle layer trained on 100 episodes, and the feature probe, which needs no model and runs once per task. Two configurations complete it where the full cross would not fit on the laptop. `configs/mac_lora.yaml` runs LoRA behavior cloning with 40 training episodes on the 24 checkpoints of at most 4B, and `configs/mac_free_reply.yaml` runs the free reply, with up to 96 new tokens, on twelve instruct or post-trained checkpoints (SmolLM2 at 360M and 1.7B, the four Qwen2.5 and the four Qwen3 sizes, and Gemma 3 at 1B and 4B) on four tasks, loan approval, support triage, gridworld and tic-tac-toe, because free replies are slow to generate. The control configuration, `configs/mac_controls.yaml`, runs Pythia and OLMo 2 with the four methods of the main sweep.

Five ablation blocks under `configs/ablations/` vary one factor at a time, on Qwen2.5-Instruct at 1.5B, 3B and 7B unless stated otherwise. The prompting block (`prompting.yaml`) adds three demonstrations, letter labels, sampling at temperature 0.7, a contextual-bandit history window of 3 instead of 10, length-normalized scoring and scoring with demonstrations, on all seven tasks. The reasoning block (`reasoning.yaml`) gives chain-of-thought a budget of 256 new tokens on the three Qwen2.5 models and compares Qwen3 at 1.7B, 4B and 8B with thinking on, also with 256 tokens, against thinking off with 16; generating 256 tokens per decision is slow on the laptop, so this block runs on gridworld, tic-tac-toe and support triage only. The adaptation block (`adaptation.yaml`) moves to the three smallest Qwen2.5-Instruct sizes, 0.5B, 1.5B and 3B, and trains probes on 10, 40 and 100 episodes, probes at the last layer and at layer 4, the feature probe, and LoRA on 10 and 40 episodes, and on 100 at 0.5B and 1.5B, on all seven tasks. The task-structure block (`tasks.yaml`) runs thirteen task configurations, among them the shifted loan variants, with generation, scoring, the probe and the feature probe, and with LoRA on 40 episodes at 1.5B and 3B.

The quantization block (`quantisation.yaml`) takes a different route, because the usual one is closed on Apple silicon. An earlier draft of this design loaded 4-bit NF4 and 8-bit weights through `bitsandbytes` (Dettmers *et al.*, 2022, 2023), which needs CUDA. We compare instead the 16-bit weights of the local backend with two GGUF quantizations of the same checkpoints: Q8_0, with 8-bit weights, and Q4_K_M, a 4-bit format that keeps some tensors at higher precision. llama.cpp serves the GGUF files with its Metal backend on the same laptop, and the harness reaches them through its OpenAI-compatible backend. That interface returns only the top log-probabilities of the first generated token, so neither the probe nor LoRA can be applied, and exact scoring becomes first-token scoring. The block therefore runs generation and first-token scoring with letter labels, at all three sizes and on all seven tasks; the 16-bit reference runs the same two conditions locally, where scoring single-token letter labels is the same as scoring their first token. H5 is restated for these formats and these two methods (Section 5.5).

Factors, their levels in the configuration files, and the hypotheses they test. “Main”, “LoRA”, “free reply” and “controls” are `configs/mac_main.yaml`, `mac_lora.yaml`, `mac_free_reply.yaml` and `mac_controls.yaml`; “ablations” are the five blocks under `configs/ablations/`.

Factor	Levels	Configuration	Tests
Parameter count	SmolLM2 135M to 1.7B; Qwen2.5 0.5B to 7B; Qwen3 0.6B to 8B; Gemma 3 270M to 4B; controls: Pythia 70M to 2.8B, OLMo 2 1B and 7B	main, controls	H1
Instruction tuning	base vs. instruct at each of the 14 sizes of the four ladders; OLMo 2 at 1B and 7B	main, LoRA, controls	H2
Conversion method	<code>prompt_generate</code> , <code>prompt_score</code> , <code>probe</code> and <code>feature_probe</code> on all 28 checkpoints; <code>lora_sft</code> on the 24 of at most 4B; <code>prompt_free</code> on 12	main, LoRA, free reply	H3, H4
Quantization	16-bit vs. GGUF Q8_0 and Q4_K_M at 1.5B, 3B and 7B, under generation and first-token letter scoring	ablations	H5
History format	bandit history as a per-arm summary vs. the raw list	ablations	H6
Reasoning budget	direct answer (16 tokens) vs. chain-of-thought (256) on Qwen2.5-Instruct; Qwen3 thinking off (16) vs. on (256) at 1.7B, 4B and 8B	ablations	H7
Prompt details	names vs. letters; 0 vs. 3 demonstrations; temperature 0 vs. 0.7; history window 10 vs. 3; raw vs. length-normalized scores; scoring with demonstrations	ablations	secondary
Adaptation data	probe on 10, 40 or 100 episodes, at the middle layer, the last layer or layer 4; LoRA on 10 or 40 episodes, and 100 at 0.5B and 1.5B	ablations	H3
Task difficulty	5 vs. 10 arms; arm gap 0.2 vs. 0.5; grid 6 vs. 9; random vs. minimax opponent; 20 vs. 40 tickets; loan shift none, covariate or sign flip	ablations	H1, H3

4.3 Seeds and data splits

The main sweep, the LoRA and control configurations and the decision-model comparison evaluate seeds 0 to 99 for every task except tic-tac-toe, which uses seeds 0 to 299 because a game’s outcome is -1 , 0 or $+1$; blackjack keeps seeds 0 to 99 but plays twenty hands per episode for the same reason. The ablations and the free-reply configuration use the first half of each range: seeds 0 to 49, 150 tic-tac-toe games, and 50 blackjack episodes of twenty hands. Probe and

LoRA training states and few-shot demonstrations come only from seeds 100000 to 100099, LoRA using the first 40 of them, and the harness rejects any configuration in which training and evaluation seeds overlap. Every model and method sees the same evaluation seeds, so all comparisons within a task configuration are paired, and an ablation cell is compared with a main-sweep cell over the seeds they share. The pilot (`configs/pilot.yaml`), in which prompts are debugged, evaluates six small checkpoints on seeds 5000 to 5019 with training states from the same training range, so no confirmatory instance is seen while prompts are being edited.

4.4 Hardware and software

Every run in this study is made on one laptop, an Apple M2 Max with 32 GB of unified memory, through PyTorch’s Metal (MPS) backend. The choice is deliberate: a machine like this is within reach of many researchers and practitioners, and a study that runs end to end on it can be checked by someone without a cluster or a cloud budget. Models run through the Transformers library ([Wolf et al., 2020](#)) with 16-bit weights, bfloat16 where Metal supports it and float16 otherwise; LoRA runs through PEFT and probes through scikit-learn ([Pedregosa et al., 2011](#)). Nothing is loaded with 4-bit or 8-bit weights through bitsandbytes, which is not available on Apple silicon. Two parts of the study run outside the local backend, on the same laptop: the quantization block, whose GGUF files llama.cpp serves (Section 4.2), and the decision models, which run in their own servers and packages (Section 4.6).

The machine sets the boundaries of the design. Exact likelihood scoring, hidden-state probes and LoRA need the weights inside the local backend, and 16-bit weights take about two bytes per parameter, about 16 GB at 8B. With activations, the operating system and the harness sharing the same unified memory, these methods stop at about 8B parameters. LoRA stops earlier, at 4B, because of training time rather than memory, and even there it is trained on 40 episodes capped at 3000 states. Section 8 lists what these limits leave out.

The run manifest records the platform string and the Python, PyTorch, Transformers, PEFT, scikit-learn and NumPy versions, so every result directory carries its own software record: [\[result pending: \[macOS, PyTorch and Transformers versions from run_manifest.json, and the llama.cpp build of the quantization block\]\]](#). Greedy generation, scoring and the probe involve no sampling, so reruns on the same laptop and library versions should agree up to the numerical nondeterminism of the Metal kernels, which we check by repeating [\[result pending: \[number of cells repeated, and the largest difference in s\]\]](#); across hardware we expect agreement within confidence intervals, not identical values.

Latencies are wall-clock times per decision on this laptop. PyTorch’s Metal backend is slower than runtimes written for Apple silicon, such as MLX or llama.cpp, so the absolute numbers understate what the same machine can do. Our latency comparisons are made within one machine and one runtime, and their ratios carry over to other settings better than their values. The quantized models in llama.cpp and the decision models in their own servers and packages share the machine but not the runtime, and their latencies are reported separately.

4.5 Statistical protocol

The unit of analysis is the episode. The primary outcome is the normalized score s of Equation (2); the secondary outcomes are the illegal-action rate, oracle agreement, the task metrics, and latency and tokens per decision. For each cell we report the mean and the interquartile mean of s over episodes and a 95% percentile bootstrap interval from 2000 resamples of episodes ([Efron and Tibshirani, 1993](#); [Agarwal et al., 2021](#)).

Two cells of the same task configuration are compared through the paired differences $\delta_\sigma = s(\pi_A, \sigma) - s(\pi_B, \sigma)$ over their shared seeds. The test statistic is $|\bar{\delta}|$, its null distribution comes from 5000 random sign flips of the δ_σ , and the two-sided p -value is $(1 + \#\{k : |\bar{\delta}^{(k)}| \geq |\bar{\delta}|\})/5001$. Effect sizes are paired Cohen’s $d = \bar{\delta}/\text{sd}(\delta)$. Within each hypothesis, the tests named in its pre-registered criterion form one family and are corrected with Holm’s step-down procedure ([Holm, 1979](#)); `deadeye compare` takes one family of pairs per call and applies the correction within it.

Scale is summarized by the least-squares fit

$$\bar{s}_c = a + b \log_{10} N_c$$

over the cells c of one ladder, method and task, where $\bar{s}_c$ is the cell mean of s and N_c the parameter count; b is the change in normalized score per tenfold increase in parameters. Each of the four ladders has three or four rungs between 135M and 8B, and `deadeye_report` fits a slope only where there are at least three. Confidence intervals for b , for differences of slopes (H1b) and for ratios of slopes (H4b) come from refitting after resampling episodes within every cell, 1000 times; `deadeye_report` writes the fits to `slopes.csv`, including a fit on the cross-task mean score per model, and `deadeye_report.slope_ratio` forms the ratios. The fit is made separately for the base and the instruct checkpoints of a ladder (`slopes.csv` carries a column for the checkpoint type), so H1 is tested on both; a ladder enters the fit only where that type has at least three rungs.

The smallest effect of interest is $\Delta = 0.10$ in s . For a paired comparison in which the per-episode differences have standard deviation σ_δ , the number of episodes needed for power $1 - \beta$ at level α is

$$n = \left\lceil \left((z_{1-\alpha/2} + z_{1-\beta}) \sigma_\delta / \Delta \right)^2 \right\rceil,$$

which `deadeye.metrics.episodes_needed` implements. With $\alpha = 0.05$ and power 0.8 this gives 32 episodes at $\sigma_\delta = 0.2$ and 126 at $\sigma_\delta = 0.4$, so the 100 episodes of the main sweep suffice when σ_δ is at most about 0.36. The ablations and the free-reply configuration evaluate 50 episodes per cell to fit the laptop’s time budget, which at the same σ_δ of 0.36 detects differences of about 0.14 rather than 0.10, and about 0.20 at $\sigma_\delta = 0.5$. We report the smallest effect each block can detect next to its results and do not read a non-significant ablation contrast as the absence of an effect. The pilot estimates of σ_δ are [\[result pending: \[per-task SD of paired differences from the pilot\]\]](#). Tasks whose anchors lie close together have a much larger per-episode spread of s . For blackjack the per-hand standard deviation of the return is close to 1, so the standard deviation of s is roughly the reciprocal of the anchor span [\[result pending: \[span \$\bar{R}_{\text{orc}} - \bar{R}_{\text{rand}}\$ on blackjack\]\]](#), which is why the sweep bundles twenty hands into each of its 100 blackjack episodes and runs 300 tic-tac-toe games: the standard deviation of an episode mean falls with the square root of the number of hands. The minimum detectable difference under this design is [\[result pending: \[per task, at power 0.8, for 100 and for 50 episodes\]\]](#), and where it exceeds 0.10 we report it next to the estimate rather than reading a non-significant result as a null.

A cell is excluded only if its model fails to load or its run crashes, never because of its outcome, and Appendix C lists every excluded cell.

4.6 Decision models

A fourth backend, `decision`, speaks the System One protocol of Jev and its open-weight relatives ([TypeSafe AI, 2026a](#); [Palmer, 2026](#); [Cloudflare, 2026](#)): the rendered task description and observation go out as the state, the legal actions as the criteria of a single choice question, and the returned probabilities are used exactly as the log-likelihoods of `prompt_score`. Table 5 lists the systems and what is verified about each from its maker’s cards, announcements and repositories; every performance figure quoted there is the maker’s own. As with the rest of the study, the comparison includes only the systems that run on the laptop: Kev at 0.8B and 4B through its own local server, Laya through its Python package, with an adapter from its interface to the protocol, and SemIf on Qwen3.5-4B, whose readout our `score_letter` condition also reproduces on the same checkpoint. The configuration `configs/decision_models.yaml` evaluates them on the seeds of the main sweep, next to the Qwen3.5 0.8B and 4B base checkpoints converted by our own methods: scoring with name labels and with letter labels, the probe, and LoRA with 40 training episodes. The Qwen2.5 and Qwen3 rows of the main sweep join them at report time. Jev is offered only as a hosted service. Clef, Clef-flash and Perplexity’s decider are served through hosted APIs, and their open weights, like Kev’s at 9B and 27B, are larger than the 8B that the local backend holds. All of them are left to future work (Section 8), and the table marks them as such. Only scoring applies to a decision model; it cannot generate, so the free-reply and generation conditions are skipped for it.

Two rows double as validations. SemIf’s readout is our letter-scoring condition, so the two should agree on the same checkpoint up to prompt wording. Kev at 0.8B and 4B sits on the same Qwen3.5 bases as our LoRA conversions and differs from them mainly in its training data, which isolates what the training target contributes, although it also reads its answer through a pointer head rather than the language-model head. Every system is reduced to the same numbers: normalized score per task with its mean and worst task, oracle agreement, calibration of the probability assigned to the chosen action against the oracle (expected calibration error and Brier score), latency per decision on the laptop (median and 95th percentile), tokens, parameters and openness.

Decision models compared with the open-weight conversions. Sizes, bases, licenses and training descriptions are from the makers’ cards and announcements. “In this study” says how a system runs on the laptop, or marks it as future work when it needs a hosted API or more memory than the laptop has. Jev’s base and training data are undisclosed.

System	Maker, license	Base and size	How it decides (as reported)	In this study
Jev (TypeSafe AI, 2026b)	TypeSafe AI; closed	undisclosed	“System One” model; reinforcement learning for calibrated decisions on synthetic data	future work (API)
Kev (Palmer, 2026)	J. Palmer; Apache-2.0	Qwen3.5 0.8B/4B/9B-Base with rank-16 LoRA and a pointer head; Qwen3.8-27B full fine-tune	trained on a 10k-example decision set plus generated policy examples; System One server	0.8B and 4B: local server; 9B and 27B: future work
Clef, Clef-flash (Cloudflare, 2026)	Cloudflare; Apache-2.0	Qwen3.8-27B, Qwen3.5-9B, frozen	routing head with low-rank adapters; Jev-compatible	future work (API)
pplx-decider v1.1 (Perplexity AI, 2026)	Perplexity; Apache-2.0 reported, gated	Qwen3.8-27B fine-tune	Decisions API with its own schema	future work (API, adapter)
Laya (Convai Innovations, 2026)	Convai Innovations; Apache-2.0	ModernBERT-large 421M; mmBERT-base 322M	encoder with a decision head; Choice, Score and Noul primitives	local package (adapter)
SemIf (Lee, 2026)	T. Lee; MIT code	frozen Qwen3 0.6B, MiniCPM5 2B, Qwen3.5-4B	option-logit readout, no training	local, on Qwen3.5-4B; also our letter scoring

This comparison is exploratory. It was added after the first draft of the pre-registration and is kept out of its tests, and we state no hypothesis about it. The decision models were trained on data we have not inspected, so contamination cannot be ruled out, and they meet our tasks through an interface designed for classification panels. What the comparison can show is whether a conversion that anyone can reproduce on open weights reaches the decision quality of purpose-built systems of similar size on tasks with a known optimum, and at what cost on the same machine.

4.7 Pre-registration

The hypotheses, predictions and falsification criteria (Appendix C) are written before the main sweep and frozen, together with the prompts and the analysis code, at the git tag `prereg-v1`, which is created after the pilot on the laptop. Prompts may change only during the pilot and only against pilot seeds. Every run after the tag is confirmatory, and any departure from the frozen design is recorded with its date and reason in the deviations table of `docs/preregistration.md` and in Appendix C. Restricting the design to one laptop was decided before the tag and before any confirmatory run, and it is the first entry of that table. [TODO: Commit hash and date of the prereg-v1 tag; deviations, if any.]

5 Results

We take the hypotheses in their pre-registered order and test each against its criterion (Appendix C). Each is tested within the scope the laptop allows: ladders from 135M to 8B, LoRA up to 4B and reasoning budgets of 256 tokens (Section 4). Unless stated otherwise, a number is a mean of s with a 95% bootstrap interval, over 100 episodes in the

main sweep (300 tic-tac-toe games; blackjack episodes of twenty hands) and over 50 in the ablations, and every comparison is paired over seeds. For each ablation result we also give the smallest difference its block can detect. Table 6 gives the main sweep by conversion method; Figure 1 shows the same cells against parameter count.

[Table placeholder: generated from the results by deadeye report] [Table placeholder: generated from the results by deadeye report] [Table placeholder: generated from the results by deadeye report] [Table placeholder: generated from the results by deadeye report]

[TODO: deadeye report writes each table as a captioned table float (pandas to_latex with caption and label); a float cannot be input inside adjustbox within this table, so the build will fail once the files exist. Emit bare tabular environments from report.py, or input the files at top level.]

5.1 H1: scale under generation

Prediction. Under prompt_generate, normalized score rises roughly linearly in the logarithm of parameter count in each of the four ladders, SmoLM2 from 135M to 1.7B, Qwen2.5 from 0.5B to 7B, Qwen3 from 0.6B to 8B and Gemma 3 from 270M to 4B, each with three or four rungs (H1a), and the slope is steeper on gridworld and tic-tac-toe than on loan approval (H1b). The prediction concerns this range and says nothing about larger models.

[Figure placeholder: generated from the results by deadeye report]

Normalized decision quality (random = 0, oracle = 1) versus parameter count, one panel per task, one line per conversion method and model family. Error bars are 95% bootstrap confidence intervals over episodes; dotted line: UCB1 on the bandit; dash-dotted line: the feature probe, which uses no language model.

Result. Under generation the fitted slope (Equation (9)) is [result pending: [b with 95% CI]] per tenfold increase in parameters for Qwen2.5, [result pending: [b, CI]] for Qwen3, [result pending: [b, CI]] for SmoLM2 and [result pending: [b, CI]] for Gemma 3. Among the controls, Pythia’s slope is [result pending: [b, CI, if the control configuration was run]]; OLMo 2, with two rungs, is plotted but not tested. H1a is therefore [result pending: [supported / falsified, naming any ladder whose interval includes 0]]. Averaged over gridworld and tic-tac-toe, the slope exceeds the loan slope by [result pending: [$b_{\text{plan}} - b_{\text{loan}}$ with CI]], and H1b is [result pending: [supported / falsified]]. Between 135M and 8B the curves are [result pending: [smooth / show a threshold near N parameters on task]]. With three or four rungs per ladder, a straight line and a gentle curve fit about equally well, so we read the slopes as summaries of the trend rather than as functional forms (Section 7). [TODO: Slopes from report/mac_main/slopes.csv, and report/mac_controls/slopes.csv for Pythia; cell means and intervals from report/mac_main/cells.csv.]

5.2 H2: instruction tuning

Prediction. At equal size, instruction-tuned checkpoints score higher than base checkpoints under prompt_generate at no fewer than 80% of the 14 sizes of the four ladders (H2a), while the instruct-minus-base gap lies within ± 0.05 under probe, at every size, and under lora_sft, at the 12 sizes of at most 4B (H2b).

Result. Under generation the instruct checkpoint leads at [result pending: [k of 4]] Qwen2.5 sizes, [result pending: [k of 4]] Qwen3 sizes, [result pending: [k of 3]] SmoLM2 sizes and [result pending: [k of 3]] Gemma 3 sizes, and at [result pending: [k of 2]] OLMo 2 sizes if the controls were run, with paired differences between [result pending: [smallest]] and [result pending: [largest difference]]; H2a is [result pending: [supported / falsified]]. [result pending: [Much / little]] of this gap is formatting: base checkpoints produce no parseable action in [result pending: [illegal-action rate, base]] of decisions, against [result pending: [illegal-action rate, instruct]] for instruct checkpoints. Under the probe the mean gap is [result pending: [gap with CI]] and under LoRA [result pending: [gap with CI]], so H2b is [result pending: [supported / falsified]]. Scoring, which also removes formatting, gives a gap of [result pending: [gap with CI]]. [TODO: Paired tests per size and task with deadeye compare, e.g. deadeye compare results/mac_main --pair "loan:Qwen__Qwen2.5-7B-Instruct/prompt_generate vs loan:Qwen__Qwen2.5-7B/prompt_generate"; LoRA pairs from results/mac_lora. pairwise_methods.csv holds only method pairs within one model. Illegal-action rates from cells.csv.]

5.3 H3: adaptation versus scale

Prediction. In distribution, the smallest rung of each ladder, fine-tuned with `lora_sft`, matches or beats the largest rung of the same ladder under `prompt_generate` on at least five of the seven tasks, in at least three of the four ladders (H3a). The pairs are SmolLM2 135M against 1.7B, Qwen2.5 0.5B against 7B, Qwen3 0.6B against 8B and Gemma 3 270M against 4B, so the zero-shot model is 12 to 15 times larger; only the small rung is fine-tuned, which keeps LoRA within its 4B limit. Under the covariate-shifted and sign-flipped loan variants, the small adapted model loses more relative to its in-distribution score than the large zero-shot model does (H3b). Those variants run in the task-structure block, where the pair is Qwen2.5-1.5B-Instruct with LoRA against Qwen2.5-7B-Instruct with generation, about five times larger.

Result. With LoRA, the smallest rung matches or beats the largest on [result pending: [*k* of 7]] tasks in SmolLM2, [result pending: [*k* of 7]] in Qwen2.5, [result pending: [*k* of 7]] in Qwen3 and [result pending: [*k* of 7]] in Gemma 3, for example [result pending: [Qwen2.5 0.5B with LoRA against Qwen2.5 7B with generation: paired difference and CI per task]]; H3a is [result pending: [supported / falsified]]. The probe on the same small rungs [result pending: [matches the large rung on *k* tasks]]. Adaptation helps most on [result pending: [tasks]] and least on [result pending: [tasks]], and in the data-budget ablation LoRA [result pending: [reaches the larger model’s score after *n* training episodes / does not reach it with 40, or with 100 at 0.5B and 1.5B]], the probe [result pending: [likewise]]. The feature probe, which uses no language model, reaches [result pending: [*s* by task]] on the same training states (Section 6.3). Under shift, the drop from the in-distribution score is [result pending: [drop with CI]] for the small adapted model and [result pending: [drop with CI]] for the large zero-shot model, an interaction of [result pending: [difference with CI]] against a minimum detectable difference of [result pending: [value]] at 50 episodes; H3b is [result pending: [supported / falsified]]. [TODO: LoRA cells from results/mac_lora and generation cells from results/mac_main; the two sides of each pair live in different runs, so pass both directories to deadeye compare. Ablation curves from report/ablations/cells.csv. The shifted loan cells train adapted methods and demonstrations on the unshifted task (train_params in configs/ablations/tasks.yaml); check summary.json (train_env_params) for every H3b cell.]

5.4 H4: scoring versus generation

Prediction. Under `prompt_score` the illegal-action rate is zero by construction (H4a), and its scale slope is less than half of the slope under `prompt_generate` (H4b).

[Figure placeholder: generated from the results by deadeye report]

Illegal or unparseable action rate under free-form generation versus parameter count.

Result. H4a holds by construction; across [result pending: [number]] scored decisions none was illegal. Under generation the illegal-action rate is [result pending: [rate]] for the smallest model and [result pending: [rate]] for the largest (Figure 2), and is highest on [result pending: [task]]. The ratio of slopes $b_{\text{score}}/b_{\text{gen}}$ is [result pending: [ratio with CI]], so H4b is [result pending: [supported / falsified]]; in Qwen2.5, scoring closes [result pending: [Z]]% of the gap between the 0.5B and the 7B model under generation. In the prompting block, letter labels change scores by [result pending: [paired difference with CI]] and length normalization by [result pending: [paired difference with CI]]. [TODO: Illegal-action rates from report/mac_main/cells.csv (illegal_rate) and tables/illegal_rate.tex; slope ratios with deadeye.report.slope_ratio on the fits in slopes.csv; generation against scoring per model from pairwise_methods.csv; format ablations from report/ablations/pairwise_methods.csv.]

5.5 H5: quantization

Prediction. Replacing 16-bit weights with GGUF Q4_K_M weights costs less than 0.05 in *s* at 7B and more than 0.10 at 1.5B, and Q8_0 weights cost less than 0.05 at every size, both under generation and under first-token scoring with letter labels. Earlier drafts stated H5 for 4-bit NF4 weights loaded through bitsandbytes; that library needs CUDA, so the prediction is restated for the GGUF formats that llama.cpp runs on the laptop and for the two methods its server supports (Section 4.2). The 3B rung is measured without a threshold.

Result. The paired 16-bit-minus-Q4_K_M difference under generation is [result pending: [difference with CI]] at 1.5B, [result pending: [difference with CI]] at 3B and [result pending: [difference with CI]] at 7B; under first-token scoring it is [result pending: [differences with CI]]. Q8_0 weights cost [result pending: [differences with CI]]. H5 is [result pending: [supported / falsified]]. With 50 episodes per cell the block detects differences of about [result pending: [minimum detectable difference]], so the 0.05 threshold [result pending: [can / cannot]] be resolved at this sample size. The quantized models run in llama.cpp and the 16-bit reference in PyTorch, so each difference also contains whatever separates the two runtimes (Section 7). [TODO: deadeye compare results/ablations/quantisation --pair "<task>:<7B, 16-bit>/prompt_generate vs <task>:<7B, Q4_K_M>/prompt_generate", per size, task and method, with the model labels of configs/ablations/quantisation.yaml; Holm over the pre-registered family. The quantized rows stay out of the scale fits.]

5.6 H6: exploration

Prediction. No model under prompting reaches the normalized score of UCB1 on the bandit (H6a), and for models below 2B a per-arm summary of the history beats the raw history (H6b).

Result. UCB1 reaches [result pending: [s with CI]] on the bandit. The best prompted cell, [result pending: [model and method]], reaches [result pending: [s with CI]], and [result pending: [number]] of [result pending: [number]] prompted cells have an upper confidence bound below UCB1's score; H6a is [result pending: [supported / falsified]]. Prompted models leave [result pending: [fraction]] of the arms unpulled on average (Section 6.1). For Qwen2.5-1.5B-Instruct, the only model below 2B in the task-structure block, the summary-minus-raw difference is [result pending: [difference with CI]], against a minimum detectable difference of [result pending: [value]] at 50 episodes, so H6b is [result pending: [supported / falsified]] for the one model it can be tested on. [TODO: Bandit cells from report/mac_main/cells.csv: norm_ci_hi of the prompted cells against norm_mean of _baseline/ucb1. The summary-versus-raw comparison pairs seeds across the bandit and bandit_rawhist configurations of results/ablations/tasks.]

5.7 H7: reasoning budget

Prediction. With a budget of 256 new tokens, chain-of-thought (Qwen2.5-Instruct at 1.5B, 3B and 7B) and thinking mode (Qwen3 at 1.7B, 4B and 8B, against thinking off with 16 tokens) improve gridworld and tic-tac-toe for the models of 4B and above, Qwen3-4B, Qwen3-8B and Qwen2.5-7B (H7a); they do not improve the models below 2B, Qwen2.5-1.5B and Qwen3-1.7B (H7b); and they lower latency-normalized score, s divided by latency per decision, at every size (H7c). The reasoning block runs on gridworld, tic-tac-toe and support triage only, and budgets of 2048 tokens are left to future work (Section 8).

Result. With chain-of-thought, scores on gridworld and tic-tac-toe change by [result pending: [mean paired difference with CI]] for Qwen2.5-7B-Instruct and by [result pending: [difference with CI]] for Qwen2.5-1.5B-Instruct. Thinking mode changes them by [result pending: [difference with CI]] for Qwen3-4B, [result pending: [difference with CI]] for Qwen3-8B and [result pending: [difference with CI]] for Qwen3-1.7B. With 50 episodes the block detects differences of about [result pending: [minimum detectable difference]]. H7a is [result pending: [supported / falsified]] and H7b is [result pending: [supported / falsified]]. Reasoning multiplies latency per decision by [result pending: [factor]], and latency-normalized score is lower with reasoning at [result pending: [number]] of [result pending: [number]] sizes; H7c is [result pending: [supported / falsified]]. On support triage, where the rules are written out in the prompt, reasoning changes scores by [result pending: [difference with CI]]. In [result pending: [fraction]] of thinking-mode decisions the reasoning used up the 256-token budget before any action appeared, so these cells measure what a short budget buys, not what thinking to completion would. [TODO: deadeye compare results/ablations/reasoning --pair "gridworld:<model>/<chain-of-thought or thinking cell> vs gridworld:<model>/prompt_generate", with each Qwen3 thinking-on cell against the thinking-off checkpoint at 16 tokens; method labels as in configs/ablations/reasoning.yaml. Latency from cells.csv (latency_per_decision_s).]

5.8 Small open-weight conversions against decision models (RQ5, exploratory)

Setting. Not pre-registered; Section 4.6 and Table 5. Kev at 0.8B and 4B, Laya, and SemIf on Qwen3.5-4B, all running on the laptop, are scored on the seven tasks and set against the same Qwen3.5 bases converted by `prompt_score`, `score_letter`, `probe` and `lora_sft`, and against the Qwen2.5 and Qwen3 rows of the main sweep. The hosted systems are not part of the comparison (Section 8). Table 7 is the complete comparison: one row per system, the same numbers for all.

[Table placeholder: generated from the results by deadeye report]

Decision quality. On the policy-following tasks (support, loan) the best decision model reaches [result pending: [s with CI]] against [result pending: [s]] for the best small conversion; on the tasks that need in-context learning or planning (contextual bandit, gridworld) the decision models score [result pending: [s]], [result pending: [above / below]] the conversions of the same bases. [TODO: Fill from report/decision_models/system_comparison.csv and the method tables, and add the per-task tables to an appendix.]

Calibration. The expected calibration error of the probability each system assigns to its chosen action is [result pending: [ECE by system, per task]]; the decision models are [result pending: [better / no better]] calibrated against a known optimum than likelihood scoring of a raw base model, and the probes trained on oracle labels reach [result pending: [ECE]]. [TODO: Tables calibration_ece and calibration_brier from the report; compare within a task only (Appendix B on tie-breaking).]

Speed and cost. On the laptop, median latency per decision is [result pending: [ms by system]], with 95th percentiles of [result pending: [ms]], and parameters and tokens per decision give [result pending: [the relative cost]]. Laya’s maker reports a few tens of milliseconds per question on a Tesla T4 (Convai Innovations, 2026); on the laptop Laya takes [result pending: [ms]]. Each decision model runs in its own server or package, so these latencies share the machine with ours but not the runtime (Section 4.4).

Validations. SemIf on Qwen3.5-4B and our letter scoring of the same checkpoint agree on [result pending: [share of decisions]] and differ in score by [result pending: [difference with CI]], which bounds the effect of prompt wording. Kev and our LoRA conversions of the same Qwen3.5 bases differ by [result pending: [difference with CI, at 0.8B and at 4B]] per task, which is mostly the contribution of training on classification corpora rather than on the task’s oracle labels.

5.9 Small versus large open-weight models: quality, speed and efficiency (RQ4)

Result. Within each ladder, latency per decision on the laptop grows [result pending: [factor]] from the smallest to the largest rung under scoring and [result pending: [factor]] under generation, while normalized score grows [result pending: [difference]]; the smallest model within [result pending: [margin]] of the largest on the cross-task mean is [result pending: [model]] under [result pending: [method]]. Tokens per decision are [result pending: [by method]], and the estimated floating-point operations per decision ($2 \times \text{parameters} \times \text{tokens}$) span [result pending: [range]]. For the unconverted assistant, on its twelve checkpoints and four tasks, the free reply takes [result pending: [latency and tokens per decision]] and contains a usable decision in [result pending: [share]] of cases, against [result pending: [latency, tokens and score difference]] for generation with a format instruction on the same checkpoints and seeds. Figure 3 shows the frontier. [TODO: From cells.csv: latency_p50_s, latency_p95_s, prompt and completion tokens, params; free-reply cells from results/mac_free_reply. Peak resident memory per cell is logged as peak_rss_mb in each summary.json and the processor name in run_manifest.json.]

[Figure placeholder: generated from the results by deadeye report]

Mean normalized score across tasks versus latency per decision on the laptop.

Figure 3 places every model and method by its mean s across tasks and its latency per decision. The frontier consists of [result pending: [cells on the frontier]]. Below [result pending: [seconds]] per decision the best cell is [result pending: [model and method]]; the best cell at any latency is [result pending: [model and method]], which is [result pending: [factor]] times slower. These latencies exclude the one-off cost of training probes and adapters, which Section 6.2 adds

back. [TODO: From report/mac_main/cells.csv (latency_per_decision_s) and report/mac_lora/cells.csv (prepare_time_s).]

6 Analysis

The hypotheses ask whether an effect exists; the per-decision logs let us ask where it comes from. The analyses below are fixed before the confirmatory runs, and every quantity in them is computed from `steps.jsonl` and `episodes.jsonl`.

6.1 Format failures and decision errors

Each decision under `prompt_generate` falls into one of four classes. A *format failure* produced no legal label and was replaced by a random action; these are the decisions logged as illegal. A *lenient parse* yielded a legal label only through one of the parser’s fallback rules, a bare label on a late line or the last label mentioned anywhere, rather than through an `Action:` line; we recover this class by re-running the parser’s stages on the logged raw output. A *decision error* is a legal action that is not optimal, and the remaining decisions are optimal. We judge optimality against the set of optimal actions, not the single action the oracle logged. Gridworld often has several shortest paths and tic-tac-toe several equally good cells, and every episode is deterministic given its seed and the actions taken, so we replay each episode, recover every visited state and compute its full optimal set.

The four rates, by model, method and task, show how much of the gap between small and large models is formatting. [result pending: [Share of the generation-minus-oracle gap attributable to format failures, by size and task]]. The comparison with scoring can be made decision by decision on loan approval, where the applicants do not depend on earlier actions, so every policy sees the same states; on the other tasks only the first decision of each episode is shared. On those shared states, generation and scoring choose the same legal action in [result pending: [agreement rate by size]] of the decisions where generation produced one. If the two agree on most such decisions, the benefit of scoring in H4 is mostly the removal of format failures; if they do not, scoring also changes which legal action the model prefers.

Chain-of-thought and thinking outputs, which the reasoning block records on gridworld, tic-tac-toe and support triage, let us look for the knowing-doing gap of Schmied *et al.* (2026). For each such decision we take the reasoning, either the text before the final `Action:` line or the `<think>` block, and check whether it names an optimal action or states the fact that makes an action optimal: an immediate win or a forced block in tic-tac-toe, the shorter way around a wall in gridworld, the rule of the written policy that applies to the ticket in support triage. We then check whether the final action differs. Automatic matching proposes candidates and a stratified sample is annotated by hand [result pending: [sample size and agreement between automatic and manual labels]]. Only the explicit `Action:` line counts as the final action here, because the parser’s last-mention rule could otherwise take the action from the reasoning itself. [result pending: [Rate of decisions with a correct rationale and a wrong action, by size and task]].

On the bandit we separate the failure modes named in prior work. Greediness is the share of arms never pulled (Schmied *et al.*, 2026); frequency bias is the probability of repeating the arm chosen most often so far; and a failure to exploit is choosing, late in an episode, an arm other than the one with the highest empirical mean among arms pulled at least k times (Harris and Slivkins, 2026), with k fixed before the confirmatory runs [TODO: choose k and add it to the pre-registration]. The suboptimal-pull fraction and the share of the last ten pulls on the best arm are logged per episode. [result pending: [The three rates by size and method]].

Errors also have a direction. In blackjack, standing where the table hits is conservative and hitting where it stands is aggressive; we tabulate both by player total, soft or hard hand, and dealer card, to see whether models are systematically risk-averse, as Jia *et al.* (2024) found language models to be in multiple-choice-list experiments [result pending: [conservative and aggressive error rates by size]]. In loan approval the analogous quantity is the approval rate relative to the oracle’s [result pending: [approval rates]]. Under the sign flip we regenerate each applicant from its seed and ask how many errors follow the usual meaning of the credit score, that is, how many decisions match what the unflipped default model would prescribe [result pending: [share of prior-following errors by size and method]].

6.2 Quality versus cost

Latency per decision mixes model size, method and hardware, so we report it next to token counts, which do not depend on hardware. All our latencies come from one laptop, and those of the main sweep from one runtime, PyTorch on Metal, which is slower than MLX or llama.cpp on the same machine (Section 4.4); ratios between cells travel better than their absolute values. Scoring needs one forward pass over the prompt and the candidate labels, generation adds one decoding step per generated token, the probe needs one forward pass and a logistic regression, and LoRA adds a one-off training cost (`prepare_time_s` in `summary.json`) to inference by scoring, a cost large enough on the laptop that we stopped LoRA at 4B. Suppose a small adapted model with latency ℓ_{small} per decision matches a larger zero-shot model with latency ℓ_{large} . Its training time T_{train} is repaid after

$$n^* = \frac{T_{\text{train}}}{\ell_{\text{large}} - \ell_{\text{small}}}$$

decisions, [\[result pending: \[\$n^*\$ for the H3 pairs that match\]\]](#). The frontier in Figure 3 leaves this training cost out; including it moves [\[result pending: \[which cells drop off the frontier\]\]](#). For H7c we divide s by latency per decision. A negative s makes that ratio hard to read, so we compare ratios only between cells whose scores are positive [\[TODO: confirm this reading of the pre-registered H7c criterion before the tag\]](#).

6.3 What the feature probe controls for

The feature probe is a control, but what it controls for depends on what the feature vector ϕ contains, and the seven tasks differ in this respect.

On loan approval the default model in Equation (5) is a logistic function of the displayed fields, linear apart from the logarithmic loan-to-income term and the cap on years of employment, and the oracle thresholds it. A linear head on ϕ can therefore come close to the oracle given enough training states, and it sets a high bar: a language-model method that does not clear it adds nothing beyond the numbers it was shown. On the shifted variants the feature probe, like every adapted method, is trained on the unshifted task. Under the sign flip it has learned the wrong sign for the credit score and should fall far below its in-distribution score; under covariate shift the default model is unchanged, but the probe must extrapolate to lower incomes and credit scores. Its drop under each shift is the reference against which H3b’s drops for language models are read.

On the contextual bandit ϕ is the current context alone, and the weights w_a are redrawn for every episode. For any fixed function of the context, each option is then equally likely to be the best one, so the expected normalized score of the feature probe is exactly zero and the control measures nothing on this task. A language-model method that does better must be using the history in its prompt, which is the in-context learning the task exists to test.

On the bandit ϕ holds pull frequencies and empirical means, and the training labels come from a clairvoyant oracle. Under the behavior policy the oracle’s arm is also the most-pulled arm, so a cloned policy can learn to repeat whichever arm it has pulled most, which is pure exploitation ([Laskin et al., 2023](#); [Nie et al., 2025](#)). The same applies to probe and `lora_sft`, whose prompts show the same counts, and to the few-shot demonstrations, whose states come from oracle rollouts, so that the only arm ever pulled in them is the oracle’s. Low bandit scores for adapted methods therefore say more about cloning a clairvoyant teacher than about the model.

On gridworld, tic-tac-toe and blackjack the oracle is a non-linear function of ϕ : a shortest path around walls, a game-tree search, and a lookup table whose stand region is not monotone in the dealer’s card. A linear head on raw features is a weak control there. The gap between probe and `feature_probe` measures how much of the decision the model’s representation makes linearly decodable, which is not the same as the model using it ([Belinkov, 2022](#)). The gap between probe and `prompt_score` on the same model measures how much the model knows but does not show ([Orgad et al., 2025](#)), and the comparison of middle and last layers tests whether that information peaks before the output ([Skean et al., 2025](#)). [\[result pending: \[Probe minus feature probe and probe minus scoring, by task and size\]\]](#). The training statistics in

summary.json, probe training accuracy and LoRA loss before and after training, separate a method that failed to fit its training states from one that fitted them and failed to generalize [result pending: [cells with low training accuracy]].

6.4 Oracle agreement and return

Oracle agreement, the share of decisions in which the policy’s own legal action equals the logged oracle action, is easy to read but has two flaws, which is why return is the primary outcome. It is a lower bound on the share of optimal decisions whenever several actions are optimal, because the oracle breaks ties by a fixed rule. In tic-tac-toe against the random opponent the oracle does not prefer quicker wins: in the state shown in Appendix A.5, four cells win with certainty, including an immediate win at cell 7, and the oracle picks cell 5. On the two bandits it also measures agreement with a clairvoyant oracle that no learner can match early in an episode; UCB1 agrees with it in [result pending: [share]] of its decisions. Fallback actions after format failures never count as agreement. For each task we report the rank correlation between agreement and s across cells [result pending: [Kendall’s τ by task]] and look at the cells where the two disagree. High agreement with low return points to errors at decisive states, such as a missed block in tic-tac-toe; low agreement with high return points to alternative optimal actions, which the set-valued optimality of Section 6.1 resolves.

7 Limitations

One laptop. Every measurement comes from one Apple M2 Max with 32 GB of memory, and the machine bounds the study. The weight-level methods, exact scoring, probes and LoRA, need the weights in the local backend, so they stop at 8B parameters, and LoRA, limited by training time, stops at 4B with 40 training episodes. Our scale slopes therefore describe the range from 135M to 8B and cannot say whether the curves bend, flatten or jump above it. The H3 pairs are 12 to 15 times apart in parameters, not the hundredfold spans that larger ladders allow, and the reasoning ablation stops at 256 tokens, a budget that Qwen3’s thinking mode may use up before it reaches an answer. Section 8 lists the rungs and conditions that did not fit. We think the restriction is a trade worth making, since anyone with similar hardware can rerun the whole study, but it is a restriction.

Synthetic tasks. The seven tasks are small: at most ten actions, at most 100 decisions per episode, and states that fit in a few lines of text. We chose them because each has an exact oracle and a distinct decision structure, not because they resemble deployed agentic work, which involves long contexts, tool outputs and open-ended actions. Slopes measured here need not transfer to such settings, and a model’s rank on DEAD EYE is evidence about the decision structures we test, not about agentic ability in general.

Oracles. Every oracle is optimal for its task, but several properties of the oracles limit what agreement and cloning can show. They break ties by fixed rules, so oracle agreement understates the share of optimal decisions wherever several moves are equally good. The blackjack oracle is optimal only for a reduced game without doubling, splitting or surrender, so our blackjack is not the casino game. The two bandit oracles are clairvoyant: a normalized score of 1 is out of reach there, and the labels used to train probes and LoRA come from a teacher whose choices cannot be inferred from the observation. A learnable teacher such as UCB1 would make behavior cloning on the bandits meaningful (Laskin *et al.*, 2023; Nie *et al.*, 2025); v1 does not use one.

Scale within families only. We draw scale conclusions only within families, and even there parameter count is confounded with the number of training tokens per parameter (Hoffmann *et al.*, 2022; Kumar *et al.*, 2025). The ladders are short, three or four rungs each, so a slope summarizes a trend more than it establishes a functional form, and its bootstrap interval reflects the noise of episodes, not the idiosyncrasies of individual checkpoints. Every rung runs with 16-bit weights, so scale is not mixed with quantization, but total parameter counts overstate the capacity of the smallest models, in which embeddings take a large share. Comparisons across families are descriptive.

Prompt sensitivity. Each condition uses one frozen prompt. Changes of format that preserve meaning can move accuracy by tens of points (Sclar *et al.*, 2024), and our format ablations, letters against names, demonstrations,

temperature and length normalization, cover only a few such changes on one ladder. We do not measure sensitivity to wording, and a different prompt might move individual cells by more than the effects we test.

Quantization. The quantization ablation measures two GGUF formats, Q8_0 and Q4_K_M, as llama.cpp runs them. It says nothing directly about the NF4 and LLM.int8() weights of bitsandbytes ([Dettmers et al., 2022, 2023](#)), which much of the work on quantized language models uses, or about GPTQ and AWQ ([Frantar et al., 2023](#); [J. Lin et al., 2024](#)). The 16-bit reference runs in PyTorch and the quantized models in llama.cpp, so each difference also contains whatever separates the two runtimes, such as kernel numerics and the rendering of the chat template; serving a 16-bit GGUF file of the same checkpoint through the same server would separate the two. Only generation and first-token letter scoring can be compared, on one ladder at three sizes.

Scoring through a server. The quantized models are served through an OpenAI-compatible interface and scored by the log-probability of the first token of each label, taken from the top 20 alternatives. This is exact only when the labels start with distinct tokens, which is why the block uses letters, and letter labels bring option-identifier priors with them ([Zheng et al., 2024](#)). These rows are reported on their own and never enter the scale curves.

Contamination. Instances are generated from seeds and never appear on the web, but the rules and strategies of tic-tac-toe and blackjack are widely published, so part of a model’s performance there may be recall of strategy rather than reasoning. The contextual bandit and the sign-flipped loan task are the cases in which prior knowledge cannot help or actively misleads.

Adaptation. Our fine-tuning is behavior cloning with LoRA, at one rank and one learning rate for every rung, on 40 training episodes in the main configuration and only up to 4B. The 7B and 8B rungs are never fine-tuned, so the LoRA half of H2b and all of H3a rest on the smaller rungs. Reinforcement-learning fine-tuning, which generalizes differently ([Chu et al., 2025](#)) and repairs some exploration failures ([Schmied et al., 2026](#)), is not part of v1, so our conclusions about small fine-tuned models concern cloning only. Out-of-distribution performance is tested on one task with two kinds of shift, which says nothing about shifts in task structure.

Statistics. One hundred episodes per cell detect a difference of 0.10 with power 0.8 only when the standard deviation of the paired differences is at most about 0.36 (Section 4.5), and the 50 episodes of the ablations and the free-reply configuration detect only about 0.14 at that spread. On tasks whose anchors lie close together, blackjack above all, power is lower even with twenty hands per episode, and a null result there is weak evidence of no effect; we report the minimum detectable difference wherever it exceeds the effect of interest. The anchors themselves are means over the evaluation seeds and we treat them as fixed, so their sampling error is not propagated into the confidence intervals of s ; the paired tests do not depend on the anchors.

Decision models. The comparison with decision models is exploratory and partial. It includes only the systems that run on the laptop, Kev at 0.8B and 4B, Laya and SemIf. Jev, whose weights, architecture and training data are closed, the hosted Clef, Clef-flash and Perplexity decider, and the larger Kev checkpoints are left out, so the comparison cannot say how the systems that defined the category fare on our tasks. The open-weight decision models were trained on classification panels, not on our tasks, so a gap in either direction may reflect training data as much as method.

Latency on one runtime. Latency depends on the hardware, the batch size and the implementation. Ours comes from PyTorch’s Metal backend, which is slower than runtimes built for Apple silicon such as MLX and llama.cpp. A model or method that is twice as fast here need not be twice as fast on a GPU or in another runtime, so our cost comparisons hold for this laptop and this runtime, and token counts are the better guide elsewhere. The quantized models and the decision models run in other runtimes on the same machine, so their latencies are not directly comparable with ours.

8 Future work: beyond one laptop

Everything left out of this study was left out because the laptop could not run it, not because we judged it unimportant. This section lists those parts in enough detail that someone with more hardware can continue where we stopped. Most of

them need no change to the harness, the tasks, the seeds or the analysis code: a new configuration file adds rows to the same report, and the scale fits accept any ladder with at least three rungs.

The upper rungs, with every method. The ladders continue above 8B: Qwen2.5 at 14B, 32B and 72B ([Yang et al., 2024](#)), Qwen3 at 14B and 32B ([Yang et al., 2025](#)) and Gemma 3 at 12B and 27B ([Gemma Team, 2025](#)). Exact scoring, probes and LoRA need their weights in accelerator memory. At about two bytes per parameter in 16-bit, one 80 GB GPU holds the rungs up to 32B; the 72B rung needs two such GPUs or 4-bit weights, which bitsandbytes NF4 loading brings to roughly 45 GB, and LoRA on 4-bit weights becomes QLoRA ([Dettmers et al., 2023](#)). The harness already loads 4-bit and 8-bit weights on CUDA. If the top rungs are quantized, the cost of quantization should be measured where both precisions fit, as the earlier form of H5 planned with NF4 and LLM.int8() weights ([Dettmers et al., 2022](#)), and the quantized points should be marked in every figure. The same hardware would extend LoRA, which stops at 4B here, to the 7B and 8B rungs, and the H3 comparison to pairs about a hundred times apart, such as Qwen2.5 0.5B against 72B.

Instruction tuning above 8B. Qwen2.5 has base and instruct checkpoints at every size, so H2 can be tested up to 72B. Qwen3 released no base checkpoint at 32B, so its base-versus-instruct comparison stops at 14B, while its post-trained 32B checkpoint still extends the H1 slope. Llama 3.1 70B Instruct ([Grattafiori, Dubey, et al., 2024](#)) would add a reference point at the top, read descriptively, since the Llama sizes come from different releases and do not form one ladder.

Larger control ladders. Pythia continues to 6.9B and 12B ([Biderman et al., 2023](#)) and OLMo 2 to 13B and 32B ([Walsh et al., 2025](#)). With these rungs the fully open ladders would span more than two orders of magnitude, and Pythia’s shared data and order would help separate parameters from training tokens. The Llama 3.x releases from 1B to 70B, gated behind their license ([Meta, 2025](#)), could serve as a further, cross-release ladder for robustness checks.

Longer reasoning. The reasoning block stops at 256 tokens, which Qwen3’s thinking mode can use up before it reaches an answer. Budgets of 2048 tokens on all seven tasks, with the separately trained Qwen3-4B Instruct-2507 and Thinking-2507 checkpoints as a second contrast, would test H7 as it was first conceived. By our planning estimates such a decision costs up to fifty times a direct answer, which is what kept it off the laptop.

Full-size ablations. The ablation blocks evaluate 50 episodes per cell and detect differences of about 0.14 rather than 0.10. Rerunning them with 100 episodes (300 tic-tac-toe games), extending the reasoning block to the four tasks it skips, and training LoRA on 100 episodes at every size would bring them to the precision of the main sweep.

Decision models at full size and as services. Kev at 9B and 27B ([Palmer, 2026](#)), Clef at 27B ([Cloudflare, 2026](#)) and Perplexity’s decider ([Perplexity AI, 2026](#)) publish weights beyond the 8B that our local backend holds. Run on one GPU machine next to our conversions, their latency would be measured on the same hardware and without a network round trip. The hosted services, Jev if access is granted ([TypeSafe AI, 2026b](#)), Clef and Clef-flash on Workers AI, and Perplexity’s Decisions API through an adapter, would complete the comparison as reference rows, with network time reported separately and no hypothesis attached, since their training data is unknown.

Reinforcement-learning conversions. DPO on pairs of oracle and model actions ([Rafailov et al., 2023](#)) and GRPO with the environment’s reward ([Shao et al., 2024](#)) would add a sixth conversion method, for which the local backend already exposes the log-probabilities and training loop. Reinforcement learning repairs some exploration failures that cloning does not ([Schmied et al., 2026](#)) and generalizes differently ([Chu et al., 2025](#)), and a learnable teacher such as UCB1 would make cloning on the bandit tasks meaningful ([Laskin et al., 2023](#)). Beyond the smallest rungs, all of this needs a GPU.

A faster backend for Apple silicon. Part of this list could return to the laptop. A backend built on MLX or llama.cpp, with access to the full logits and hidden states that an OpenAI-compatible server does not expose, would make exact scoring and probes faster on the same machine and, with 4-bit weights, would fit rungs of 14B and perhaps 32B into its memory.

Cloud routes and their cost. Three routes provide the hardware, at costs we can only estimate. Kaggle’s free T4 hours, a weekly quota on 16 GB GPUs, are enough for some of the smaller additions, such as longer reasoning budgets on the

small rungs or 4-bit probes up to about 14B. A rented A100 or H100 with 80 GB costs a few US dollars per hour, and the GPU sweep that earlier drafts of this design planned, with the ladders up to 72B and every method, was estimated at roughly 150 GPU-hours, a few hundred dollars at those rates. For prompting alone, a hosted OpenAI-compatible provider would be cheaper still, but the provider then controls precision and sampling and neither probes nor LoRA can be applied, so such rows could only be descriptive and would stay out of the scale fits.

9 Conclusion

We built DEAD EYE to separate three things that evaluations of language-model agents usually run together: how large the model is, how it is turned into a policy, and what kind of decision it faces. We ran the whole study on one laptop, so that anyone with similar hardware can check it. [\[result pending: \[Two or three sentences on the main findings: the slope under generation and how it differs between planning and one-shot tasks; whether adaptation lets the smallest rung of a ladder match one 12 to 15 times larger, and where it breaks under shift; how much of the small-model deficit scoring recovers.\]\]](#) For someone choosing a model for a decision task, the practical reading is [\[result pending: \[recommendation on model size and conversion method, conditional on the findings\]\]](#). The next steps follow from the limitations, and Section 8 sets them out: the rungs above 8B with every method, longer reasoning budgets, reinforcement-learning conversions, learnable teachers for the bandit tasks, and longer text environments in which the oracle is harder to compute.

Reproducibility statement. The harness, configurations, prompts and analysis code are released under the Apache-2.0 license at the repository linked on the title page, and the pre-registration is frozen at the git tag `prereg-v1`. Every run uses one Apple M2 Max laptop with 32 GB of memory. Every result table and figure is produced by `deadeye report` from logged runs, and every number in the text is taken from the report’s `cells.csv` or `pairwise_methods.csv` or from `deadeye compare`. Appendix A reproduces every prompt verbatim, Appendix B the task generators, Appendix C the pre-registered criteria, Appendix D the time each configuration takes on the laptop and Appendix E the commands that rebuild the paper from a fresh clone. [\[TODO: Archive DOI for the code, results and report.\]](#)

10 Prompts

This appendix reproduces the messages a model receives, as printed by `deadeye show-prompt` for seed 0 after two oracle steps. The text is verbatim with two exceptions made for typesetting. Each system message is a single line in the prompt and is wrapped here at word boundaries, and the chain-of-thought instruction, also a single line, is wrapped the same way. Lines reading `[[user]]` and `[[assistant]]` mark the start of a chat message and are not part of the prompt. The system message is the same in every variant of a task, so it is shown once per task.

A few details are easier to state than to show. The probe embeds the generation-mode prompt, shown as the default variant, and LoRA trains and acts with the scoring-mode prompt. The prompts use each environment’s default parameters, which match the main sweep except for gridworld: the sweep’s grid is 6×6 , so its system message reads `6x6 grid` and at most 24 moves and its move counter (`max 24`). For blackjack at seed 0 the hand ends with the second oracle action, a stand, so the state shown is the decision after one hit. In the tic-tac-toe state shown, cells 5, 6, 7 and 9 all win with certainty and the oracle picks cell 5, although cell 7 wins at once (Section 6.4). Few-shot demonstrations come from training seeds, so in tic-tac-toe they can show the agent playing the other mark, as the second demonstration below does; for the shifted loan variants they come from the unshifted task. When demonstrations are combined with chain-of-thought, each demonstration’s assistant turn carries the fixed rationale `Choosing the best available option.` before its action line. Base checkpoints receive the same messages without a chat template: the system text, then each user turn after a line `Input:` and each assistant turn after a line `Output:`, separated by blank lines and ending with a line `Output:`.

10.1 Multi-armed bandit (bandit)

System message (identical in every variant; a single line in the prompt, wrapped here):

```
You are playing a 5-armed bandit for 50 rounds. Each round you choose one arm and receive a reward of 1 or 0. Each arm has a fixed but unknown success probability. Your goal is to maximise the total reward over all rounds, which requires balancing exploration (trying arms to learn about them) and exploitation (choosing the arm that looks best).
```

Default: free-form generation, action names. Command: `deadeye show-prompt bandit --seed 0 --steps 2;`
oracle action in this state: `arm_5`.

```
[[user]]
Round 3 of 50.
Summary of past pulls:
  arm_1: never pulled
  arm_2: never pulled
  arm_3: never pulled
  arm_4: never pulled
  arm_5: pulled 2 times, average reward 0.00
Legal actions: arm_1, arm_2, arm_3, arm_4, arm_5
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

Letter action format. Command: `deadeye show-prompt bandit --seed 0 --steps 2 --action-format letter;`
oracle action in this state: `arm_5`.

```
[[user]]
Round 3 of 50.
Summary of past pulls:
  arm_1: never pulled
  arm_2: never pulled
  arm_3: never pulled
  arm_4: never pulled
  arm_5: pulled 2 times, average reward 0.00
Legal actions: A, B, C, D, E
  A = arm_1
  B = arm_2
  C = arm_3
  D = arm_4
  E = arm_5
Reply with exactly one line of the form `Action: <letter>` and nothing else.
```

Chain of thought. Command: `deadeye show-prompt bandit --seed 0 --steps 2 --cot;` oracle action in this state: `arm_5`.

```
[[user]]
Round 3 of 50.
Summary of past pulls:
  arm_1: never pulled
  arm_2: never pulled
  arm_3: never pulled
  arm_4: never pulled
  arm_5: pulled 2 times, average reward 0.00
Legal actions: arm_1, arm_2, arm_3, arm_4, arm_5
Think step by step in at most three short sentences, then end your reply with a final line of the form `Action: <name>`.
```

Likelihood scoring (also the LoRA training and inference prompt). Command: `deadeye show-prompt bandit --seed 0 --steps 2 --method prompt_score`; oracle action in this state: `arm_5`.

```
[[user]]
Round 3 of 50.
Summary of past pulls:
  arm_1: never pulled
  arm_2: never pulled
  arm_3: never pulled
  arm_4: never pulled
  arm_5: pulled 2 times, average reward 0.00
Legal actions: arm_1, arm_2, arm_3, arm_4, arm_5
Answer with only the action name of your chosen action.
```

Two oracle demonstrations from training seeds. Command: `deadeye show-prompt bandit --seed 0 --steps 2 --few-shot 2`; oracle action in this state: `arm_5`.

```
[[user]]
Round 4 of 50.
Summary of past pulls:
  arm_1: pulled 3 times, average reward 1.00
  arm_2: never pulled
  arm_3: never pulled
  arm_4: never pulled
  arm_5: never pulled
Legal actions: arm_1, arm_2, arm_3, arm_4, arm_5
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: arm_1
[[user]]
Round 2 of 50.
Summary of past pulls:
  arm_1: never pulled
  arm_2: never pulled
  arm_3: pulled 1 times, average reward 1.00
  arm_4: never pulled
  arm_5: never pulled
Legal actions: arm_1, arm_2, arm_3, arm_4, arm_5
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: arm_3
[[user]]
Round 3 of 50.
Summary of past pulls:
  arm_1: never pulled
  arm_2: never pulled
  arm_3: never pulled
  arm_4: never pulled
  arm_5: pulled 2 times, average reward 0.00
Legal actions: arm_1, arm_2, arm_3, arm_4, arm_5
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

10.2 Contextual bandit (contextual_bandit)

System message (identical in every variant; a single line in the prompt, wrapped here):

```
You are making 30 sequential decisions. Each round you see a context made of 4 numeric
features
```

(x1..x4) and must choose one of 3 options (option_A, option_B, option_C). Each option's success probability depends on the context through a fixed but unknown rule. You receive a reward of 1 (success) or 0 (failure). Use the history of past contexts, choices and rewards to infer which option works best for which contexts, and maximise total reward.

Default: free-form generation, action names. Command: `deadeye show-prompt contextual_bandit --seed 0 -steps 2`; oracle action in this state: `option_B`.

```
[[user]]
Round 3 of 30.
Past rounds (most recent last, showing up to 10):
  context [x1=-2.33, x2=-0.22, x3=-1.25, x4=-0.73] -> chose option_C -> reward 1
  context [x1=-0.54, x2=-0.32, x3=+0.41, x4=+1.04] -> chose option_B -> reward 1
Current context: [x1=-0.13, x2=+1.37, x3=-0.67, x4=+0.35]
Legal actions: option_A, option_B, option_C
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

Letter action format. Command: `deadeye show-prompt contextual_bandit --seed 0 --steps 2 --action-format letter`; oracle action in this state: `option_B`.

```
[[user]]
Round 3 of 30.
Past rounds (most recent last, showing up to 10):
  context [x1=-2.33, x2=-0.22, x3=-1.25, x4=-0.73] -> chose option_C -> reward 1
  context [x1=-0.54, x2=-0.32, x3=+0.41, x4=+1.04] -> chose option_B -> reward 1
Current context: [x1=-0.13, x2=+1.37, x3=-0.67, x4=+0.35]
Legal actions: A, B, C
  A = option_A
  B = option_B
  C = option_C
Reply with exactly one line of the form `Action: <letter>` and nothing else.
```

Chain of thought. Command: `deadeye show-prompt contextual_bandit --seed 0 --steps 2 --cot`; oracle action in this state: `option_B`.

```
[[user]]
Round 3 of 30.
Past rounds (most recent last, showing up to 10):
  context [x1=-2.33, x2=-0.22, x3=-1.25, x4=-0.73] -> chose option_C -> reward 1
  context [x1=-0.54, x2=-0.32, x3=+0.41, x4=+1.04] -> chose option_B -> reward 1
Current context: [x1=-0.13, x2=+1.37, x3=-0.67, x4=+0.35]
Legal actions: option_A, option_B, option_C
Think step by step in at most three short sentences, then end your reply with a final line of the form `Action: <name>`.
```

Likelihood scoring (also the LoRA training and inference prompt). Command: `deadeye show-prompt contextual_bandit --seed 0 --steps 2 --method prompt_score`; oracle action in this state: `option_B`.

```
[[user]]
Round 3 of 30.
Past rounds (most recent last, showing up to 10):
  context [x1=-2.33, x2=-0.22, x3=-1.25, x4=-0.73] -> chose option_C -> reward 1
  context [x1=-0.54, x2=-0.32, x3=+0.41, x4=+1.04] -> chose option_B -> reward 1
Current context: [x1=-0.13, x2=+1.37, x3=-0.67, x4=+0.35]
```

```
Legal actions: option_A, option_B, option_C
Answer with only the action name of your chosen action.
```

Two oracle demonstrations from training seeds. Command: `deadeye show-prompt contextual_bandit --seed 0 --steps 2 --few-shot 2`; oracle action in this state: `option_B`.

```
[[user]]
Round 4 of 30.
Past rounds (most recent last, showing up to 10):
  context [x1=+0.25, x2=-1.16, x3=-0.21, x4=-0.16] -> chose option_A -> reward 1
  context [x1=-0.15, x2=+1.38, x3=-0.34, x4=-0.48] -> chose option_B -> reward 1
  context [x1=+0.63, x2=+0.54, x3=+0.61, x4=+1.11] -> chose option_C -> reward 1
Current context: [x1=+1.48, x2=+0.03, x3=-1.14, x4=-0.58]
Legal actions: option_A, option_B, option_C
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: option_B
[[user]]
Round 2 of 30.
Past rounds (most recent last, showing up to 10):
  context [x1=-0.82, x2=+0.71, x3=-0.75, x4=+0.41] -> chose option_A -> reward 1
Current context: [x1=-0.10, x2=-0.93, x3=+2.11, x4=+0.72]
Legal actions: option_A, option_B, option_C
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: option_C
[[user]]
Round 3 of 30.
Past rounds (most recent last, showing up to 10):
  context [x1=-2.33, x2=-0.22, x3=-1.25, x4=-0.73] -> chose option_C -> reward 1
  context [x1=-0.54, x2=-0.32, x3=+0.41, x4=+1.04] -> chose option_B -> reward 1
Current context: [x1=-0.13, x2=+1.37, x3=-0.67, x4=+0.35]
Legal actions: option_A, option_B, option_C
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

10.3 Loan approval (loan)

System message (identical in every variant; a single line in the prompt, wrapped here):

```
You are a loan officer deciding whether to approve loan applications. For each applicant you
see
their profile and the loan terms. If you approve and the loan is repaid, the bank earns the
interest shown; if you approve and the applicant defaults, the bank loses the amount shown.
Denying earns and loses nothing. Approve an application only when you expect it to be
profitable
on average, using the applicant's creditworthiness. Your goal is to maximise total profit.
```

Default: free-form generation, action names. Command: `deadeye show-prompt loan --seed 0 --steps 2`; oracle action in this state: `approve`.

```
[[user]]
Applicant 3 of 20.
  Age: 41
  Annual income: $67.6k
  Existing debt-to-income ratio: 0.43
  Credit score: 752 (range 300-850)
  Years in current employment: 2
```

```
Requested loan: $9.6k for car
If repaid the bank earns $1.2k; if the applicant defaults the bank loses $5.8k.
Legal actions: approve, deny
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

Letter action format. Command: `deadeye show-prompt loan --seed 0 --steps 2 --action-format letter;`
oracle action in this state: approve.

```
[[user]]
Applicant 3 of 20.
  Age: 41
  Annual income: $67.6k
  Existing debt-to-income ratio: 0.43
  Credit score: 752 (range 300-850)
  Years in current employment: 2
  Requested loan: $9.6k for car
  If repaid the bank earns $1.2k; if the applicant defaults the bank loses $5.8k.
Legal actions: A, B
  A = approve
  B = deny
Reply with exactly one line of the form `Action: <letter>` and nothing else.
```

Chain of thought. Command: `deadeye show-prompt loan --seed 0 --steps 2 --cot;` oracle action in this state: approve.

```
[[user]]
Applicant 3 of 20.
  Age: 41
  Annual income: $67.6k
  Existing debt-to-income ratio: 0.43
  Credit score: 752 (range 300-850)
  Years in current employment: 2
  Requested loan: $9.6k for car
  If repaid the bank earns $1.2k; if the applicant defaults the bank loses $5.8k.
Legal actions: approve, deny
Think step by step in at most three short sentences, then end your reply with a final line of
the form `Action: <name>`.
```

Likelihood scoring (also the LoRA training and inference prompt). Command: `deadeye show-prompt loan --seed 0 --steps 2 --method prompt_score;` oracle action in this state: approve.

```
[[user]]
Applicant 3 of 20.
  Age: 41
  Annual income: $67.6k
  Existing debt-to-income ratio: 0.43
  Credit score: 752 (range 300-850)
  Years in current employment: 2
  Requested loan: $9.6k for car
  If repaid the bank earns $1.2k; if the applicant defaults the bank loses $5.8k.
Legal actions: approve, deny
Answer with only the action name of your chosen action.
```

Two oracle demonstrations from training seeds. Command: `deadeye show-prompt loan --seed 0 --steps 2 --few-shot 2;` oracle action in this state: approve.

```

[[user]]
Applicant 4 of 20.
  Age: 54
  Annual income: $41.1k
  Existing debt-to-income ratio: 0.34
  Credit score: 727 (range 300-850)
  Years in current employment: 0
  Requested loan: $6.8k for debt consolidation
  If repaid the bank earns $0.8k; if the applicant defaults the bank loses $4.1k.
Legal actions: approve, deny
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: approve
[[user]]
Applicant 2 of 20.
  Age: 34
  Annual income: $40.3k
  Existing debt-to-income ratio: 0.20
  Credit score: 686 (range 300-850)
  Years in current employment: 23
  Requested loan: $8.6k for small business
  If repaid the bank earns $1.0k; if the applicant defaults the bank loses $5.2k.
Legal actions: approve, deny
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: approve
[[user]]
Applicant 3 of 20.
  Age: 41
  Annual income: $67.6k
  Existing debt-to-income ratio: 0.43
  Credit score: 752 (range 300-850)
  Years in current employment: 2
  Requested loan: $9.6k for car
  If repaid the bank earns $1.2k; if the applicant defaults the bank loses $5.8k.
Legal actions: approve, deny
Reply with exactly one line of the form `Action: <name>` and nothing else.

```

10.4 Gridworld navigation (gridworld)

System message (identical in every variant; a single line in the prompt, wrapped here):

```

You are navigating a 5x5 grid. 'A' marks your position, 'G' the goal, '#' walls and '.' free
cells. Rows are numbered top to bottom and columns left to right, both starting at 0. Each
move
goes one cell up, down, left or right; you cannot move into walls or off the grid. Reach the
goal in as few moves as possible (at most 20 moves).

```

Default: free-form generation, action names. Command: `deadeye show-prompt gridworld --seed 0 --steps 2;`
oracle action in this state: left.

```

[[user]]
Move 3 (max 20).
Map:
.#...
#.G..
...#.
...A#

```

```
#....
You are at row 3, column 3. The goal is at row 1, column 2.
Legal actions: down, left
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

Letter action format. Command: `deadeye show-prompt gridworld --seed 0 --steps 2 --action-format letter`; oracle action in this state: left.

```
[[user]]
Move 3 (max 20).
Map:
.#...
#.G..
...#.
...A#
#....
You are at row 3, column 3. The goal is at row 1, column 2.
Legal actions: A, B
  A = down
  B = left
Reply with exactly one line of the form `Action: <letter>` and nothing else.
```

Chain of thought. Command: `deadeye show-prompt gridworld --seed 0 --steps 2 --cot`; oracle action in this state: left.

```
[[user]]
Move 3 (max 20).
Map:
.#...
#.G..
...#.
...A#
#....
You are at row 3, column 3. The goal is at row 1, column 2.
Legal actions: down, left
Think step by step in at most three short sentences, then end your reply with a final line of
the form `Action: <name>`.
```

Likelihood scoring (also the LoRA training and inference prompt). Command: `deadeye show-prompt gridworld --seed 0 --steps 2 --method prompt_score`; oracle action in this state: left.

```
[[user]]
Move 3 (max 20).
Map:
.#...
#.G..
...#.
...A#
#....
You are at row 3, column 3. The goal is at row 1, column 2.
Legal actions: down, left
Answer with only the action name of your chosen action.
```

Two oracle demonstrations from training seeds. Command: `deadeye show-prompt gridworld --seed 0 --steps 2 --few-shot 2`; oracle action in this state: left.

```

[[user]]
Move 4 (max 20).
Map:
.....
...A.
#.#G
.....
.#.#.
You are at row 1, column 3. The goal is at row 2, column 4.
Legal actions: up, left, right
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: right
[[user]]
Move 2 (max 20).
Map:
.A...
#.#..
..#..
..#.G
..#..
You are at row 0, column 1. The goal is at row 3, column 4.
Legal actions: down, left, right
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: right
[[user]]
Move 3 (max 20).
Map:
.#...
#.G..
...#.
...A#
#....
You are at row 3, column 3. The goal is at row 1, column 2.
Legal actions: down, left
Reply with exactly one line of the form `Action: <name>` and nothing else.

```

10.5 Tic-tac-toe (tictactoe)

System message (identical in every variant; a single line in the prompt, wrapped here):

```

You are playing tic-tac-toe as 'X' against an opponent playing 'O'. The board cells are
numbered
1 to 9 from left to right, top to bottom. Three of your marks in a row, column or diagonal
wins.
Choose the number of an empty cell. Win if you can, otherwise force a draw; never lose if
avoidable.

```

Default: free-form generation, action names. Command: `deadeye show-prompt tictactoe --seed 0 --steps 2;`
oracle action in this state: 5.

```

[[user]]
You are 'X'. Board (numbers mark empty cells):
X 2 0
X 5 6
7 0 9

```

Legal actions: 2, 5, 6, 7, 9
Reply with exactly one line of the form `Action: <name>` and nothing else.

Letter action format. Command: `deadeye show-prompt tictactoe --seed 0 --steps 2 --action-format letter`; oracle action in this state: 5.

```
[[user]]
You are 'X'. Board (numbers mark empty cells):
X 2 0
X 5 6
7 0 9
Legal actions: A, B, C, D, E
  A = 2
  B = 5
  C = 6
  D = 7
  E = 9
Reply with exactly one line of the form `Action: <letter>` and nothing else.
```

Chain of thought. Command: `deadeye show-prompt tictactoe --seed 0 --steps 2 --cot`; oracle action in this state: 5.

```
[[user]]
You are 'X'. Board (numbers mark empty cells):
X 2 0
X 5 6
7 0 9
Legal actions: 2, 5, 6, 7, 9
Think step by step in at most three short sentences, then end your reply with a final line of
the form `Action: <name>`.
```

Likelihood scoring (also the LoRA training and inference prompt). Command: `deadeye show-prompt tictactoe --seed 0 --steps 2 --method prompt_score`; oracle action in this state: 5.

```
[[user]]
You are 'X'. Board (numbers mark empty cells):
X 2 0
X 5 6
7 0 9
Legal actions: 2, 5, 6, 7, 9
Answer with only the action name of your chosen action.
```

Two oracle demonstrations from training seeds. Command: `deadeye show-prompt tictactoe --seed 0 --steps 2 --few-shot 2`; oracle action in this state: 5.

```
[[user]]
You are 'X'. Board (numbers mark empty cells):
1 2 3
4 5 6
7 8 9
Legal actions: 1, 2, 3, 4, 5, 6, 7, 8, 9
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: 1
[[user]]
You are 'O'. Board (numbers mark empty cells):
1 2 3
```

```

4 0 6
X 8 X
Legal actions: 1, 2, 3, 4, 6, 8
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: 8
[[user]]
You are 'X'. Board (numbers mark empty cells):
X 2 0
X 5 6
7 0 9
Legal actions: 2, 5, 6, 7, 9
Reply with exactly one line of the form `Action: <name>` and nothing else.

```

10.6 Blackjack (blackjack)

System message (identical in every variant; a single line in the prompt, wrapped here):

```

You are playing blackjack against the dealer. Cards 2-10 are worth their number, face cards
are
worth 10 and an ace is worth 11 if that does not bust you, otherwise 1. You may 'hit' (take
another card) or 'stand' (end your turn). If you exceed 21 you bust and lose. After you
stand,
the dealer draws until reaching 17 or more. You win if your total is higher than the dealer's
or
the dealer busts; equal totals tie. A two-card 21 pays extra. Maximise your expected
winnings.

```

Default: free-form generation, action names. Command: `deadeye show-prompt blackjack --seed 0 --steps 2`; oracle action in this state: stand.

```

[[user]]
Your cards: 7, 2, 10 (total 19).
Dealer shows: 3.
Legal actions: hit, stand
Reply with exactly one line of the form `Action: <name>` and nothing else.

```

Letter action format. Command: `deadeye show-prompt blackjack --seed 0 --steps 2 --action-format letter`; oracle action in this state: stand.

```

[[user]]
Your cards: 7, 2, 10 (total 19).
Dealer shows: 3.
Legal actions: A, B
  A = hit
  B = stand
Reply with exactly one line of the form `Action: <letter>` and nothing else.

```

Chain of thought. Command: `deadeye show-prompt blackjack --seed 0 --steps 2 --cot`; oracle action in this state: stand.

```

[[user]]
Your cards: 7, 2, 10 (total 19).
Dealer shows: 3.
Legal actions: hit, stand

```

Think step by step in at most three short sentences, then end your reply with a final line of the form `Action: <name>`.

Likelihood scoring (also the LoRA training and inference prompt). Command: `deadeye show-prompt blackjack --seed 0 --steps 2 --method prompt_score`; oracle action in this state: stand.

```
[[user]]
Your cards: 7, 2, 10 (total 19).
Dealer shows: 3.
Legal actions: hit, stand
Answer with only the action name of your chosen action.
```

Two oracle demonstrations from training seeds. Command: `deadeye show-prompt blackjack --seed 0 --steps 2 --few-shot 2`; oracle action in this state: stand.

```
[[user]]
Your cards: 8, 5 (total 13).
Dealer shows: 2.
Legal actions: hit, stand
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: stand
[[user]]
Your cards: 6, 10 (total 16).
Dealer shows: 7.
Legal actions: hit, stand
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: hit
[[user]]
Your cards: 7, 2, 10 (total 19).
Dealer shows: 3.
Legal actions: hit, stand
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

10.7 Customer-support triage (support)

System message (identical in every variant; the policy text is part of it):

You are a customer support agent deciding how to resolve tickets. For each ticket you see the customer's message and the order facts. Apply the company policy exactly:

1. If the customer has 3 or more prior claims and the issue is a damaged item, a wrong item or a missing delivery, escalate to the fraud team.
2. Billing errors: refund.
3. Changed mind: refund if the item was delivered at most 14 days ago (30 days for premium customers), otherwise deny.
4. Late delivery (item received): `partial_refund` if it was more than 7 days late, otherwise deny.
5. Not received: if it is at most 3 days past the promised date, deny (ask the customer to wait); if later and the order value is over \$100, escalate; otherwise refund.
6. Damaged or wrong item: if no photo evidence is attached and the order value is over \$50, `request_evidence`; otherwise refund if the customer asks for a refund, else replace if a replacement is in stock, else refund.

Correct decisions earn credit; unwarranted payouts cost the order value and refusing a legitimate claim loses the customer.

Default: free-form generation, action names. Command: `deadeye show-prompt support --seed 0 --steps 2;`
oracle action in this state: `escalate`.

```
[[user]]
Ticket 3 of 10.
Customer message: "Wrong item in the box. I don't want it, please refund."
Order: backpack, value $59.79. Delivered 53 days ago.
Photo evidence attached: no. Prior claims by this customer: 3. Customer tier: standard.
  Replacement in stock: yes.
Legal actions: refund, replace, partial_refund, request_evidence, escalate, deny
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

Free reply (the unconverted assistant). Command: `deadeye show-prompt support --seed 0 --steps 2 --method prompt_free;` oracle action in this state: `escalate`.

```
[[user]]
Ticket 3 of 10.
Customer message: "Wrong item in the box. I don't want it, please refund."
Order: backpack, value $59.79. Delivered 53 days ago.
Photo evidence attached: no. Prior claims by this customer: 3. Customer tier: standard.
  Replacement in stock: yes.
Legal actions: refund, replace, partial_refund, request_evidence, escalate, deny
What do you do? Answer in your own words.
```

Letter action format. Command: `deadeye show-prompt support --seed 0 --steps 2 --action-format letter;` oracle action in this state: `escalate`.

```
[[user]]
Ticket 3 of 10.
Customer message: "Wrong item in the box. I don't want it, please refund."
Order: backpack, value $59.79. Delivered 53 days ago.
Photo evidence attached: no. Prior claims by this customer: 3. Customer tier: standard.
  Replacement in stock: yes.
Legal actions: A, B, C, D, E, F
  A = refund
  B = replace
  C = partial_refund
  D = request_evidence
  E = escalate
  F = deny
Reply with exactly one line of the form `Action: <letter>` and nothing else.
```

Chain of thought. Command: `deadeye show-prompt support --seed 0 --steps 2 --cot;` oracle action in this state: `escalate`.

```
[[user]]
Ticket 3 of 10.
Customer message: "Wrong item in the box. I don't want it, please refund."
Order: backpack, value $59.79. Delivered 53 days ago.
Photo evidence attached: no. Prior claims by this customer: 3. Customer tier: standard.
  Replacement in stock: yes.
Legal actions: refund, replace, partial_refund, request_evidence, escalate, deny
Think step by step in at most three short sentences, then end your reply with a final line of
the form `Action: <name>`.
```

Likelihood scoring (also the LoRA training and inference prompt). Command: `deadeye show-prompt support --seed 0 --steps 2 --method prompt_score`; oracle action in this state: `escalate`.

```
[[user]]
Ticket 3 of 10.
Customer message: "Wrong item in the box. I don't want it, please refund."
Order: backpack, value $59.79. Delivered 53 days ago.
Photo evidence attached: no. Prior claims by this customer: 3. Customer tier: standard.
Replacement in stock: yes.
Legal actions: refund, replace, partial_refund, request_evidence, escalate, deny
Answer with only the action name of your chosen action.
```

Two oracle demonstrations from training seeds. Command: `deadeye show-prompt support --seed 0 --steps 2 --few-shot 2`; oracle action in this state: `escalate`.

```
[[user]]
Ticket 4 of 10.
Customer message: "I don't want the backpack anymore. It's unused."
Order: backpack, value $46.76. Delivered 54 days ago.
Photo evidence attached: yes. Prior claims by this customer: 1. Customer tier: premium.
Replacement in stock: yes.
Legal actions: refund, replace, partial_refund, request_evidence, escalate, deny
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: deny
[[user]]
Ticket 2 of 10.
Customer message: "There's a wrong charge on my phone case order, please fix it."
Order: phone case, value $67.26. Delivered 8 days ago.
Photo evidence attached: no. Prior claims by this customer: 2. Customer tier: standard.
Replacement in stock: yes.
Legal actions: refund, replace, partial_refund, request_evidence, escalate, deny
Reply with exactly one line of the form `Action: <name>` and nothing else.
[[assistant]]
Action: refund
[[user]]
Ticket 3 of 10.
Customer message: "Wrong item in the box. I don't want it, please refund."
Order: backpack, value $59.79. Delivered 53 days ago.
Photo evidence attached: no. Prior claims by this customer: 3. Customer tier: standard.
Replacement in stock: yes.
Legal actions: refund, replace, partial_refund, request_evidence, escalate, deny
Reply with exactly one line of the form `Action: <name>` and nothing else.
```

11 Task parameters and generators

Table 8 lists every task parameter with its default, its value in the main sweep and its values in the ablations. The rest of this appendix gives the generators in full.

Task parameters. Ablation labels are the task-configuration keys used in the ablation blocks under `configs/ablations/`; blank cells take the main-sweep value. Every ablation evaluates 50 episodes per task configuration, 150 for tic-tac-toe.

Task	Parameter	Default	Main sweep	Ablation variants
bandit	n_arms	5	5	10 (bandit_hard)
	horizon	50	50	100 (bandit_hard)
	gap	0.2	0.2	0.5 (bandit_easy)

Task	Parameter	Default	Main sweep	Ablation variants
	means_mode	gap	gap	
	history_format	summary	summary	raw(bandit_rawhist)
contextual_bandit	n_actions	3	3	
	dim	4	4	
	horizon	30	30	
	history_window	10	10	
	weight_scale	2.0	2.0	
loan	n_applicants	20	20	
	interest_rate	0.12	0.12	
	loss_given_default	0.6	0.6	
	shift	none	none	covariate(loan_covariate_shift), sign_flip(loan_sign_flip); both trained on none
gridworld	size	5	6	9 (gridworld_large)
	n_walls	size	6	16
	max_steps	4×size	24	36
	step_penalty	0.02	0.02	
	min_distance	size	6	9
tictactoe	opponent	random	random	minimax(tictactoe_minimax)
	agent_mark	alternate	alternate	
	episodes		300	150
blackjack	natural_bonus	true	true	
	hands_per_episode	1	20	20
	episodes		100	50
support	n_tickets	10	20	40 (support_long)

Bandit. With means_mode set to gap, the index of the best arm is drawn uniformly and the reward table, T rows by K columns of Bernoulli draws, follows from the same generator; the alternative uniform mode, which draws each μ_a from $U(0.1, 0.9)$, is not used in our configurations. The pseudo-regret is computed from the true means, not from the realized rewards. The feature vector has $2K$ entries: each arm’s share of the pulls so far and its empirical mean, with 0.5 for unpulled arms. An episode has exactly T decisions.

Contextual bandit. The generator draws the weight matrix ($A \times d$, entries from $\mathcal{N}(0, 2^2)$), then the $T \times d$ contexts (entries from $\mathcal{N}(0, 1)$, rounded to two decimals), then the $T \times A$ table of Bernoulli outcomes. Regret uses the true success probabilities. The feature vector is the current context. An episode has exactly T decisions.

Loan approval. For each applicant the generator draws the annual income in thousands of dollars, $I = \text{clip}(e^Z, 12, 400)$ with $Z \sim \mathcal{N}(\ln 55, 0.5^2)$ and rounded to one decimal; the credit score, $C = \lfloor \text{clip}(\mathcal{N}(680, 70^2), 300, 850) \rfloor$; the age, uniform on $\{21, \dots, 70\}$; the debt-to-income ratio, $D = \text{clip}(\text{Beta}(2, 5), 0, 0.9)$ rounded to two decimals; the years in current employment, $E = \lfloor \text{clip}(\text{Exp}(\text{mean } 6), 0, 35) \rfloor$; the requested amount in thousands, $L = \text{clip}(e^{Z'}, 1, 150)$ with $Z' \sim \mathcal{N}(\ln 15, 0.6^2)$ and rounded to one decimal; and a purpose drawn uniformly from car, home improvement, small business, education and debt consolidation, with offsets $\rho = -0.2, -0.3, +0.6, 0$ and $+0.4$ in Equation (5).

Defaults are then drawn as Bernoulli(p_i) for all applicants at once. Covariate shift replaces **ln 55** by **ln 35** and the credit mean 680 by 600 and leaves the default model unchanged; the sign flip sets $c = +1$ and leaves the applicant distribution unchanged, so a sign-flipped seed shows the same applicants as the unshifted one. The feature vector holds age, income, debt ratio, credit score, years employed and amount, standardized as $(x - \mu)/s$ with (μ, s) equal to (45, 15), (55, 40), (0.28, 0.15), (680, 70), (6, 6) and (15, 15), followed by a one-hot purpose. An episode has exactly N decisions.

Gridworld. The generator permutes the n^2 cells; the first becomes the start, the second the goal and the next n_walls become walls. It redraws until breadth-first search from the goal reaches the start at distance $d^* \geq \text{min_distance}$. The feature vector holds the agent’s and the goal’s row and column divided by n , followed by the n^2 wall indicators. An episode has between d^* and $4n$ decisions.

Tic-tac-toe. The agent plays X on even seeds and moves first, and O on odd seeds, in which case the opponent opens. Both opponents draw from a generator seeded with $\sigma + 10007$: the random opponent chooses uniformly among empty cells, and the minimax opponent chooses uniformly among the moves of highest depth-aware minimax value from its own point of view. The feature vector holds the nine cells as $+1$ for the agent’s marks, -1 for the opponent’s and 0 for empty ones. An episode has between two and five decisions.

Blackjack. A generator seeded with $\sigma + 20011$ draws two streams of 32 cards, one for the player and one for the dealer, each card uniform over the 13 entries $\{1, \dots, 9, 10, 10, 10, 10\}$. The player receives the first two cards of the player stream and the dealer the first two of the dealer stream, and the dealer’s first card is shown. The dealer’s hand is not checked for a natural before the player acts, and a player’s two-card 21 against a dealer total of 21 is a tie. The feature vector is the hand total divided by 21, the soft-hand flag and the dealer’s card divided by 10. An episode has at least one decision and is capped at 12.

Customer-support triage. A generator seeded with $\sigma + 30013$ draws, for each of the n tickets, the issue type uniformly from six (damaged, not received, wrong item, late, changed mind, billing error), what the customer asks for (refund with probability 0.45, replacement 0.25, nothing specific 0.30), the item from a list of eight, the order value $\text{clip}(e^Z, 5, 500)$ with $Z \sim \mathcal{N}(\ln 60, 0.8^2)$, the days since delivery (uniform on 0 to 60), the days past the promised date (uniform on 0 to 14), whether photo evidence is attached (probability 0.5), the number of prior claims (from $\{0, 0, 0, 1, 1, 2, 3, 4\}$), the customer tier (premium with probability 0.3) and whether a replacement is in stock (probability 0.7). The customer message is one of two or three fixed sentences per issue and request, with the item name filled in; one of them is the familiar “I received the package damaged. I need a refund today.” The six-rule policy in the task description is the oracle. A correct decision earns $+1$; a wrong one -1 , less the payout in hundreds of dollars when the agent refunds, replaces (or half of it for a partial refund) where the policy said deny, request evidence or escalate, and less a further 1 when it denies a claim the policy would have paid. The feature vector is the one-hot issue and request, the scaled timing, value, evidence, prior-claim, tier and stock fields.

Other sources of randomness. The random baseline uses one generator per cell, seeded with 0. When a decision is illegal, the substitute action comes from a generator seeded with $\sigma + 7$ for that episode. The behavior policy that collects probe and LoRA training states uses one generator, seeded with 0, across all training episodes. Few-shot demonstrations take one state from each training seed in order, starting at 100000; a generator seeded with 12345 draws the number of oracle steps taken before the demonstration state uniformly from 0 to $\min(5, H) - 1$, and if the episode ends during those steps the initial state is used instead. Greedy decoding uses no randomness; for sampled decoding the model’s generator is seeded with the episode seed before each episode.

12 Pre-registered predictions and falsification criteria

This appendix restates docs/preregistration.md as frozen at the tag prereg-v1. **[TODO: Re-copy after the tag is created, and record any deviation below.]**

Research questions. RQ1: how does decision quality scale with parameter count within a model family, and does the slope depend on the task’s decision structure? RQ2: does the conversion method (prompting, likelihood scoring, hidden-state probe, LoRA behavior cloning) change the dependence on scale? RQ3: which other factors (instruction tuning, quantization, reasoning budget, observation format, prompt encoding, adaptation data budget) move decision quality, and by how much? RQ4: what is the frontier of quality against inference cost? RQ5, exploratory and not a pre-registered test: how do small open-weight models converted here compare with the purpose-built decision models that run on the same laptop (Kev at 0.8B and 4B, Laya and SemIf) in decision quality, calibration, latency and cost on the same tasks and seeds? The hosted decision models are future work.

Predictions. Table 9 lists every prediction with its criteria. “Score” is the normalized score s ; “CI” is a 95% bootstrap confidence interval; slopes are the b of Equation (9), fitted within one ladder on the main-sweep results.

Pre-registered predictions with the conditions under which each is supported or falsified. Outcomes between the two conditions are reported as inconclusive.

	Prediction	Supported if	Falsified if
H1a	Under <code>prompt_generate</code> , $b > 0$ in every ladder: SmolLM2 135M to 1.7B, Qwen2.5 0.5B to 7B, Qwen3 0.6B to 8B, Gemma 3 270M to 4B	the CI of b excludes 0 and is positive for every ladder with at least three rungs	any ladder’s CI includes 0 or is negative
H1b	The slope is larger for gridworld and tic-tac-toe than for loan	the bootstrap CI of $b_{\text{plan}} - b_{\text{loan}}$ excludes 0	that CI includes 0
H2a	Instruct beats base under <code>prompt_generate</code> at every size of the four ladders	the CI of the paired difference lies above 0 at 80% of sizes or more	it does so at fewer than 50% of sizes
H2b	The instruct-minus-base gap under <code>probe</code> (every size) and <code>lora_sft</code> (sizes up to 4B) is below 0.05	the CI of the mean gap lies within $[-0.05, 0.05]$	the CI excludes that interval
H3a	The smallest rung of each ladder with <code>lora_sft</code> is at least as good as its largest rung with <code>prompt_generate</code> , 12 to 15 times larger (SmolLM2 135M and 1.7B, Qwen2.5 0.5B and 7B, Qwen3 0.6B and 8B, Gemma 3 270M and 4B), in distribution	the CI of the paired difference is at or above 0 on at least 5 of 7 tasks, in at least 3 of the 4 ladders	this holds on 3 or fewer tasks in 2 or more ladders
H3b	On <code>loan_sign_flip</code> and <code>loan_covariate_shift</code> , the small adapted model loses more relative to in distribution than the large model (task-structure block: Qwen2.5-Instruct 1.5B adapted, 7B zero-shot)	the CI of the interaction excludes 0	that CI includes 0
H4a	The illegal-action rate under <code>prompt_score</code> is 0	by construction	not applicable
H4b	$b_{\text{score}} < 0.5 b_{\text{gen}}$	the bootstrap CI of the ratio lies below 0.5	the CI includes 0.5 or lies above it
H5	GGUF Q4_K_M weights cost less than 0.05 at 7B and more than 0.10 at 1.5B, and Q8_0 weights less than 0.05 at every size, under generation and first-token letter scoring	the CIs of the paired 16-bit-minus-quantized differences meet the thresholds	any threshold fails
H6a	No model under prompting reaches the UCB1 score on the bandit	every CI lies below the UCB1 score	any CI includes or exceeds it
H6b	Summary history beats raw history for models below 2B	the paired CI lies above 0 for every such model	any such CI includes 0
H7a	A 256-token thinking or chain-of-thought budget improves gridworld and tic-tac-toe at 4B and above (Qwen3 4B and 8B, Qwen2.5 7B)	the paired CI lies above 0	the CI includes 0
H7b	The same budget does not improve models below 2B (Qwen3 1.7B, Qwen2.5 1.5B)	the paired CI includes 0 or is negative	the CI lies above 0
H7c	Latency-normalized score is lower with the budget at every size	the ratio of score to latency is lower at every size	it is higher at any size

Design, analysis and exclusions. Tasks, parameters and seeds are those of the configuration files at the tag, run on one Apple M2 Max with 32 GB of memory and 16-bit weights, with LoRA on the rungs of at most 4B trained on 40 episodes

and at most 3000 states. Evaluation seeds are 0 to 99 in the main sweep (0 to 299 for tic-tac-toe; blackjack plays twenty hands per episode) and 0 to 49 in the ablations and the free-reply configuration (0 to 149 for tic-tac-toe); training seeds are 100000 to 100099, and prompts are frozen at the tag. One hundred paired episodes detect a difference of 0.10 with power 0.8 when the standard deviation of the paired differences is at most 0.36; fifty detect about 0.14 at that spread, and the smallest detectable effect is reported next to every ablation result. The primary outcome is the normalized return per episode; secondary outcomes are the illegal-action rate, oracle agreement, the task metrics, and latency and tokens per decision. Each cell is summarized by its mean, interquartile mean and percentile bootstrap interval (2000 resamples); paired comparisons use sign-flip permutation tests (5000 permutations) with Holm’s correction within each hypothesis’s family of tests; slopes are fitted by ordinary least squares with bootstrap intervals over episodes. No episode is excluded because of its outcome. A cell is excluded only if its model fails to load or its run crashes, as recorded in `run_manifest.json`, and no interim analysis changes the design.

Deviations and excluded cells. One change was made before the tag and before any confirmatory run. On 9 October 2026 the scope was fixed to one Apple M2 Max laptop: ladders up to 8B with LoRA up to 4B, GGUF quantization through `llama.cpp` in place of `bitsandbytes`, reasoning budgets of 256 tokens, local decision models only and ablations of 50 episodes, with everything else moved to future work, because the study has no GPU budget. [TODO: Copy any later deviations from docs/preregistration.md (date, change, reason) and list every excluded cell with the reason recorded in run_manifest.json; write “none” if there are none.]

13 Compute budget

Every configuration runs on one Apple M2 Max laptop with 32 GB of memory. The times in this appendix are planning estimates, not measurements. The pilot measures the latency per decision on the laptop, [result pending: [seconds per decision by size and method, from the pilot]], and the estimates will be replaced by the measured times. [TODO: Replace Table 10 with measured laptop-hours per configuration, summed from eval_time_s and prepare_time_s in summary.json.]

Planning estimates of laptop time per configuration, to be replaced by measured times.

Configuration	What it runs	Laptop-hours
<code>pilot.yaml</code>	six small checkpoints, shortened tasks, seeds 5000 to 5019	a few
<code>mac_main.yaml</code>	28 checkpoints; generation, scoring, probe and feature probe; seven tasks	150 to 200
<code>mac_lora.yaml</code>	LoRA on the 24 checkpoints of at most 4B, 40 training episodes	50 to 80
<code>mac_free_reply.yaml</code>	free reply on twelve instruct checkpoints, four tasks, 50 episodes	30 to 40
<code>ablations/*.yaml</code>	five blocks, 50 episodes per cell	about 100
<code>decision_models.yaml</code>	Kev, Laya and SemIf; the Qwen3.5 0.8B and 4B base checkpoints converted	about 10
<code>mac_controls.yaml</code>	Pythia and OLMo 2, if time allows	about 30
Total	without the pilot and the controls	340 to 430

The order of the runs is chosen so that stopping early still leaves usable results. The pilot comes first and is followed by the `prereg-v1` tag. The main sweep then runs overnight for two to three weeks, ladder by ladder in the order Qwen2.5, Qwen3, SmolLM2 and Gemma 3, so that a run stopped at any point leaves complete ladders behind it; the runner skips finished cells, so each night picks up where the last one stopped. LoRA, the free replies, the five ablation blocks and the decision models follow, and the controls come last, if time allows.

Counted with `deadeye` estimate, which rolls out the oracle on the first ten seeds of each task configuration, one model and method in the main sweep make 5000 decisions on the bandit, 3000 on the contextual bandit, 2000 each on loan approval and support triage, about 660 on gridworld, 930 on tic-tac-toe and 3170 on blackjack, about 16800 in all. Weaker policies take more steps on gridworld, so these counts are lower bounds there. With 28 checkpoints and three language-model methods (the feature probe runs once, without a model), the main sweep makes about 1.4 million

language-model decisions before probe training; `deadeye estimate`, which multiplies by all four method configurations, reports about 1.9 million. In the planning estimates, chain-of-thought or thinking with 256 new tokens costs roughly ten times as much per decision as a direct answer, which is why the reasoning block runs on three tasks only. LoRA training takes [\[result pending: \[minutes per cell by model size on the laptop\]\]](#). In memory, 16-bit weights need about 2 bytes per parameter, about 16 GB at 8B, plus a small key-value cache for these short prompts.

14 Reproducibility checklist

The commands below rebuild every result in the paper from a fresh clone on an Apple-silicon Mac; the same commands run on a CUDA machine or, slowly, on a CPU. Every step can be stopped and resumed, since the runner skips cells whose summary already exists, which is how the long configurations run over many nights.

```
git clone https://github.com/CaptiveON/DeadEye && cd DeadEye
git checkout prereg-v1 # frozen prompts, configs and analysis code
python3 -m venv .venv && source .venv/bin/activate
pip install torch # macOS arm64 wheels include Metal (MPS)
pip install -e ".[hf,dev]" tabulate
pytest -q # unit and integration tests, no downloads
deadeye smoke # every task and method, mock and tiny random model
deadeye show-prompt gridworld --seed 3 --cot --few-shot 2
deadeye estimate configs/mac_main.yaml --seconds-per-decision 0.5
deadeye run configs/pilot.yaml # pilot, seeds 5000-5019; then tag prereg-v1
caffeinate -i deadeye run configs/mac_main.yaml # main sweep, resumable
deadeye run configs/mac_lora.yaml # LoRA on the checkpoints of at most 4B
deadeye run configs/mac_free_reply.yaml # free replies: 12 checkpoints, 4 tasks
deadeye run configs/ablations/prompting.yaml # then each other file in configs/ablations/;
# quantisation.yaml needs llama-server running
deadeye run configs/decision_models.yaml # with Kev, Laya and SemIf running locally
deadeye run configs/mac_controls.yaml # optional: Pythia and OLMo 2
deadeye report results/mac_main --out report/mac_main
deadeye compare results/mac_main \
  --pair "loan:Qwen__Qwen2.5-1.5B-Instruct/prompt_score vs loan:Qwen__Qwen2.5-1.5B-
  Instruct/prompt_generate"
cd paper && make assets && make # regenerate tables and figures, build the PDF
```

- **Code and license.** The harness, configurations, prompts and analysis code are released under Apache-2.0; a Dockerfile builds a CPU image with the harness installed, whose entry point is the `deadeye` command.
- **Hardware.** One Apple M2 Max with 32 GB of unified memory, PyTorch’s Metal backend and 16-bit weights; llama.cpp with its Metal backend for the GGUF files of the quantization block. [\[TODO: Hardware and library versions as in Section 4.4.\]](#)
- **Access.** The Gemma 3 checkpoints are gated behind their terms of use, which must be accepted on the Hugging Face hub before the main sweep. The quantization block needs the GGUF files and one llama.cpp server per file, and the decision-model comparison needs Kev’s local server, the Laya package and SemIf running on the same machine.
- **Tests.** `pytest` checks that oracles return legal actions and episodes terminate, that a seed reproduces its instance, and that the oracle beats the random policy on every task. It checks the gridworld oracle against shortest paths; that the tic-tac-toe oracle never loses to the minimax opponent and that expectimax does at least as well as minimax against the random one; the blackjack table on known hands; and that the dealer’s cards do not depend on the player’s hits. On the model side it checks exact scoring against a manual log-probability computation, including labels that are token prefixes of one another, that greedy generation is deterministic and ignores checkpoint repetition penalties, that hidden states differ by layer, that LoRA training changes the model, and that

base checkpoints are rendered without a chat template. It also covers the parser on realistic outputs, the prompt structure, the illegal-action policies and the rule that fallback actions never count as agreement, the rejection of overlapping training and evaluation seeds, the selection of training variants, the runner end to end, and the report on smoke-test results.

- **Determinism.** Instances, reward tables and card streams are fixed by the seed. Greedy decoding, scoring and probes involve no sampling and should reproduce on the same machine and library versions, up to the numerical nondeterminism of the Metal kernels (Section 4.4).
- **Provenance.** Every cell’s `summary.json` records the model and method specification, precision, parameter count, seeds and timings, and `run_manifest.json` the configuration, the platform and the library versions (Section 3.6).
- **Released data.** `summary.json` and `episodes.jsonl` for every cell, the per-decision `steps.jsonl` as an archive, and the generated report. [\[TODO: Archive DOI.\]](#)
- **Exclusions and deviations.** Listed in Appendix C.

15 Model ladders: verified repository identifiers

Table 11 lists the Hugging Face repository identifiers of the checkpoints in this study as `docs/model_ladders.md` records them; an identifier counts as verified when it was seen exactly in an official listing. Cells marked “not verified” are configured checkpoints whose identifiers follow the family’s naming pattern but have not been confirmed. The decision-model comparison adds the Qwen3.5 0.8B and 4B base checkpoints, `Qwen/Qwen3.5-0.8B-Base` and `Qwen/Qwen3.5-4B-Base`, neither of them verified yet. Qwen3 also has separately trained instruct and thinking checkpoints at 4B, `Qwen/Qwen3-4B-Instruct-2507` and `Qwen/Qwen3-4B-Thinking-2507`, both verified, which offer a second contrast for H7 that this study does not run. [\[TODO: Verify the remaining identifiers before the sweep: the Qwen2.5 1.5B and 7B base checkpoints, the Qwen3 0.6B, 1.7B and 4B base and post-trained checkpoints, the SmoLLM2 135M and 360M base and 1.7B Instruct checkpoints, the two Gemma 3 270M checkpoints and the two Qwen3.5 bases.\]](#)

Verified repository identifiers of the checkpoints in this study. For Pythia, the non-deduplicated identifiers (without `-deduped`) are verified as well, and Pythia has no instruct checkpoints. The larger rungs (Qwen2.5 14B to 72B, Qwen3 14B and 32B, Gemma 3 12B and 27B, Pythia 6.9B and 12B, OLMo 2 13B and 32B) are future work (Section 8); `docs/model_ladders.md` lists their identifiers.

Family	Size	Base or pre-trained	Instruct or post-trained
Qwen2.5	0.5B	<code>Qwen/Qwen2.5-0.5B</code>	<code>Qwen/Qwen2.5-0.5B-Instruct</code>
	1.5B	not verified	<code>Qwen/Qwen2.5-1.5B-Instruct</code>
	3B	<code>Qwen/Qwen2.5-3B</code>	<code>Qwen/Qwen2.5-3B-Instruct</code>
	7B	not verified	<code>Qwen/Qwen2.5-7B-Instruct</code>
Qwen3	0.6B, 1.7B, 4B	not verified	not verified
	8B	<code>Qwen/Qwen3-8B-Base</code>	<code>Qwen/Qwen3-8B</code>
SmoLLM2	135M	not verified	<code>HuggingFaceTB/SmoLLM2-135M-Instruct</code>
	360M	not verified	<code>HuggingFaceTB/SmoLLM2-360M-Instruct</code>
	1.7B	<code>HuggingFaceTB/SmoLLM2-1.7B</code>	not verified
Gemma 3	270M	not verified	not verified
	1B	<code>google/gemma-3-1b-pt</code>	<code>google/gemma-3-1b-it</code>
	4B	<code>google/gemma-3-4b-pt</code>	<code>google/gemma-3-4b-it</code>
Pythia	70M	<code>EleutherAI/pythia-70m-deduped</code>	
	160M	<code>EleutherAI/pythia-160m-deduped</code>	

Family	Size	Base or pre-trained	Instruct or post-trained
	410M	EleutherAI/pythia-410m-deduped	
	1B	EleutherAI/pythia-1b-deduped	
	1.4B	EleutherAI/pythia-1.4b-deduped	
	2.8B	EleutherAI/pythia-2.8b-deduped	
OLMo 2	1B	allenai/OLMo-2-0425-1B	allenai/OLMo-2-0425-1B-Instruct
	7B	allenai/OLMo-2-1124-7B	allenai/OLMo-2-1124-7B-Instruct

References

- Agarwal, R. *et al.* (2021) “Deep reinforcement learning at the edge of the statistical precipice,” *Advances in neural information processing systems 34 (NeurIPS 2021)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2108.13264>.
- Auer, P., Cesa-Bianchi, N. and Fischer, P. (2002) “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, 47, pp. 235–256. Available at: <https://doi.org/10.1023/A:1013689704352>.
- Baldwin, R.R. *et al.* (1956) “The optimum strategy in blackjack,” *Journal of the American Statistical Association*, 51(275), pp. 429–439. Available at: <https://doi.org/10.1080/01621459.1956.10501334>.
- Belinkov, Y. (2022) “Probing classifiers: Promises, shortcomings, and advances,” *Computational Linguistics*, 48(1), pp. 207–219. Available at: https://doi.org/10.1162/coli_a_00422.
- Ben Allal, L. *et al.* (2025) “SmolLM2: When smol goes big – data-centric training of a small language model,” *arXiv preprint arXiv:2502.02737* [Preprint]. Available at: <https://arxiv.org/abs/2502.02737>.
- Biderman, S. *et al.* (2023) “Pythia: A suite for analyzing large language models across training and scaling,” *Proceedings of the 40th international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 2397–2430. Available at: <https://arxiv.org/abs/2304.01373>.
- Binz, M. *et al.* (2025) “A foundation model to predict and capture human cognition,” *Nature*, 644(8078), pp. 1002–1009. Available at: <https://doi.org/10.1038/s41586-025-09215-4>.
- Binz, M. and Schulz, E. (2023) “Using cognitive psychology to understand GPT-3,” *Proceedings of the National Academy of Sciences*, 120(6), p. e2218523120. Available at: <https://doi.org/10.1073/pnas.2218523120>.
- Brown, T.B. *et al.* (2020) “Language models are few-shot learners,” *Advances in neural information processing systems 33 (NeurIPS 2020)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2005.14165>.
- Carta, T. *et al.* (2023) “Grounding large language models in interactive environments with online reinforcement learning,” *Proceedings of the 40th international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 3676–3713. Available at: <https://arxiv.org/abs/2302.02662>.
- Chu, T. *et al.* (2025) “SFT memorizes, RL generalizes: A comparative study of foundation model post-training,” *Proceedings of the 42nd international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 10818–10838. Available at: <https://arxiv.org/abs/2501.17161>.
- Cloudflare (2026) “Introducing Clef: Our open-source decision models, and new RL fine-tuning platform.” Cloudflare blog, 1 October 2026, <https://blog.cloudflare.com/clef-decision-models/>.
- Convai Innovations (2026) “Laya: Open System One decision models (model cards for laya, laya-multilingual and laya-typed-decisions).” <https://huggingface.co/convaiinnovations/laya>; <https://huggingface.co/convaiinnovations/laya-typed-decisions>; <https://laya.convaiinnovations.com/>.
- Dettmers, T. *et al.* (2022) “GPT3.int8(): 8-bit matrix multiplication for transformers at scale,” *Advances in neural information processing systems 35 (NeurIPS 2022)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2208.07339>.
- Dettmers, T. *et al.* (2023) “QLoRA: Efficient finetuning of quantized LLMs,” *Advances in neural information processing systems 36 (NeurIPS 2023)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2305.14314>.
- Dettmers, T. and Zettlemoyer, L. (2023) “The case for 4-bit precision: K-bit inference scaling laws,” *Proceedings of the 40th international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 7750–7774. Available at: <https://arxiv.org/abs/2212.09720>.
- Duan, J. *et al.* (2024) “GTBench: Uncovering the strategic reasoning capabilities of LLMs via game-theoretic evaluations,” *Advances in neural information processing systems 37 (NeurIPS 2024)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2402.12348>.

- Efron, B. and Tibshirani, R.J. (1993) *An introduction to the bootstrap*. New York: Chapman & Hall (Monographs on statistics and applied probability, 57). Available at: <https://doi.org/10.1201/9780429246593>.
- Fan, C. *et al.* (2024) “Can large language models serve as rational players in game theory? A systematic analysis,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16), pp. 17960–17967. Available at: <https://doi.org/10.1609/aaai.v38i16.29751>.
- Frantar, E. *et al.* (2023) “OPTQ: Accurate quantization for generative pre-trained transformers,” *The eleventh international conference on learning representations (ICLR 2023)*. OpenReview.net. Available at: <https://arxiv.org/abs/2210.17323>.
- Gema, A.P. *et al.* (2025) “Inverse scaling in test-time compute,” *Transactions on Machine Learning Research* [Preprint]. Available at: <https://openreview.net/forum?id=NXgyHW1c7M>.
- Gemma Team (2025) “Gemma 3 technical report,” *arXiv preprint arXiv:2503.19786* [Preprint]. Available at: <https://arxiv.org/abs/2503.19786>.
- Grattafiori, A., Dubey, A., *et al.* (2024) “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783* [Preprint]. Available at: <https://arxiv.org/abs/2407.21783>.
- Guo, D. *et al.* (2025) “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning,” *Nature*, 645(8081), pp. 633–638. Available at: <https://doi.org/10.1038/s41586-025-09422-z>.
- Harris, K. and Slivkins, A. (2026) “Should you use your large language model to explore or exploit?” *Proceedings of the 42nd conference on uncertainty in artificial intelligence*. PMLR (Proceedings of machine learning research), pp. 2008–2058. Available at: <https://arxiv.org/abs/2502.00225>.
- Hegselmann, S. *et al.* (2023) “TabLLM: Few-shot classification of tabular data with large language models,” *Proceedings of the 26th international conference on artificial intelligence and statistics*. PMLR (Proceedings of machine learning research), pp. 5549–5581. Available at: <https://arxiv.org/abs/2210.10723>.
- Henderson, P. *et al.* (2018) “Deep reinforcement learning that matters,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). Available at: <https://doi.org/10.1609/aaai.v32i1.11694>.
- Hoffmann, J. *et al.* (2022) “An empirical analysis of compute-optimal large language model training,” *Advances in neural information processing systems 35 (NeurIPS 2022)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2203.15556>.
- Holm, S. (1979) “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, 6(2), pp. 65–70.
- Holtzman, A. *et al.* (2021) “Surface form competition: Why the highest probability answer isn’t always right,” *Proceedings of the 2021 conference on empirical methods in natural language processing*. Online; Punta Cana, Dominican Republic: Association for Computational Linguistics, pp. 7038–7051. Available at: <https://doi.org/10.18653/v1/2021.emnlp-main.564>.
- Hu, E.J. *et al.* (2022) “LoRA: Low-rank adaptation of large language models,” *The tenth international conference on learning representations (ICLR 2022)*. OpenReview.net. Available at: <https://arxiv.org/abs/2106.09685>.
- Huang, W. *et al.* (2022) “Language models as zero-shot planners: Extracting actionable knowledge for embodied agents,” *Proceedings of the 39th international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 9118–9147. Available at: <https://arxiv.org/abs/2201.07207>.
- Jia, J. *et al.* (2024) “Decision-making behavior evaluation framework for LLMs under uncertain context,” *Advances in neural information processing systems 37 (NeurIPS 2024)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2406.05972>.
- Jiwatode, M. *et al.* (2026) “Spatial reasoning in LLM game agents: Impact of causal context and multi-step planning,” *arXiv preprint arXiv:2607.22732* [Preprint]. Available at: <https://arxiv.org/abs/2607.22732>.
- Krishnamurthy, A. *et al.* (2024) “Can large language models explore in-context?” *Advances in neural information processing systems 37 (NeurIPS 2024)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2403.15371>.
- Kumar, T. *et al.* (2025) “Scaling laws for precision,” *The thirteenth international conference on learning representations (ICLR 2025)*. OpenReview.net. Available at: <https://arxiv.org/abs/2411.04330>.
- Laskin, M. *et al.* (2023) “In-context reinforcement learning with algorithm distillation,” *The eleventh international conference on learning representations (ICLR 2023)*. OpenReview.net. Available at: <https://arxiv.org/abs/2210.14215>.
- Lattimore, T. and Szepesvári, C. (2020) *Bandit algorithms*. Cambridge: Cambridge University Press. Available at: <https://doi.org/10.1017/9781108571401>.
- Law, A.M. (2015) *Simulation modeling and analysis*. Fifth. New York: McGraw-Hill Education.
- Lee, T. (2026) “SemIf (formerly OpenJev): Semantic ifs from open models.” Software, <https://github.com/TheoLeeCJ/SemIf-OpenJev>.
- Li, Y. *et al.* (2025) “Small models struggle to learn from strong reasoners,” *Findings of the association for computational linguistics: ACL 2025*. Vienna, Austria: Association for Computational Linguistics, pp. 25366–25394. Available at: <https://doi.org/10.18653/v1/2025.findings-acl.1301>.

- Lin, B.Y. *et al.* (2024) “The unlocking spell on base LLMs: Rethinking alignment via in-context learning,” *The twelfth international conference on learning representations (ICLR 2024)*. OpenReview.net. Available at: <https://arxiv.org/abs/2312.01552>.
- Lin, J. *et al.* (2024) “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” *Proceedings of machine learning and systems 6 (MLSys 2024)*. Available at: <https://arxiv.org/abs/2306.00978>.
- Liu, B. *et al.* (2026) “SPIRAL: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning,” *The fourteenth international conference on learning representations (ICLR 2026)*. OpenReview.net. Available at: <https://arxiv.org/abs/2506.24119>.
- Liu, R., Geng, J., Peterson, J.C., *et al.* (2025) “Large language models assume people are more rational than we really are,” *The thirteenth international conference on learning representations (ICLR 2025)*. OpenReview.net. Available at: <https://arxiv.org/abs/2406.17055>.
- Liu, R., Geng, J., Wu, A.J., *et al.* (2025) “Mind your step (by step): Chain-of-thought can reduce performance on tasks where thinking makes humans worse,” *Proceedings of the 42nd international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 38489–38517. Available at: <https://arxiv.org/abs/2410.21333>.
- Liu, X. *et al.* (2024) “AgentBench: Evaluating LLMs as agents,” *The twelfth international conference on learning representations (ICLR 2024)*. OpenReview.net. Available at: <https://arxiv.org/abs/2308.03688>.
- Meta (2025) “Llama models.” Official GitHub repository meta-llama/llama-models (model cards, release table and licences for Llama 3.1, 3.2, 3.3 and 4). Available at: <https://github.com/meta-llama/llama-models>.
- Monea, G. *et al.* (2025) “LLMs are in-context bandit reinforcement learners,” *Second conference on language modeling (COLM 2025)*. OpenReview.net. Available at: <https://arxiv.org/abs/2410.05362>.
- Nie, A. *et al.* (2025) “EVOLvE: Evaluating and optimizing LLMs for in-context exploration,” *Proceedings of the 42nd international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 46346–46376. Available at: <https://arxiv.org/abs/2410.06238>.
- Orgad, H. *et al.* (2025) “LLMs know more than they show: On the intrinsic representation of LLM hallucinations,” *The thirteenth international conference on learning representations (ICLR 2025)*. OpenReview.net. Available at: <https://arxiv.org/abs/2410.02707>.
- Paglieri, D. *et al.* (2025) “BALROG: Benchmarking agentic LLM and VLM reasoning on games,” *The thirteenth international conference on learning representations (ICLR 2025)*. OpenReview.net. Available at: <https://arxiv.org/abs/2411.13543>.
- Palmer, J. (2026) “Kev: Open decision models on Qwen3.5 and Qwen3.8 with a System One-compatible server.” Software and model cards, <https://github.com/jaredpalmer/kev>; <https://huggingface.co/jaredpalmer/kev-9b>.
- Pedregosa, F. *et al.* (2011) “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, 12(85), pp. 2825–2830.
- Perplexity AI (2026) “Pplx-decider-v1.1-27b: Model card.” <https://huggingface.co/perplexity-ai/pplx-decider-v1.1-27b>.
- Pomerleau, D.A. (1988) “ALVINN: An autonomous land vehicle in a neural network,” in D.S. Touretzky (ed.) *Advances in neural information processing systems 1 (NIPS 1988)*. Morgan Kaufmann, pp. 305–313.
- Rafailov, R. *et al.* (2023) “Direct preference optimization: Your language model is secretly a reward model,” *Advances in neural information processing systems 36 (NeurIPS 2023)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2305.18290>.
- Robinson, J. and Wingate, D. (2023) “Leveraging large language models for multiple choice question answering,” *The eleventh international conference on learning representations (ICLR 2023)*. OpenReview.net. Available at: <https://arxiv.org/abs/2210.12353>.
- Ross, S., Gordon, G. and Bagnell, D. (2011) “A reduction of imitation learning and structured prediction to no-regret online learning,” *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. Fort Lauderdale, FL, USA: PMLR (Proceedings of machine learning research), pp. 627–635. Available at: <https://arxiv.org/abs/1011.0686>.
- Ruoss, A. *et al.* (2025) “LMAct: A benchmark for in-context imitation learning with long multimodal demonstrations,” *Proceedings of the 42nd international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 52303–52344. Available at: <https://arxiv.org/abs/2412.01441>.
- Schaeffer, R., Miranda, B. and Koyejo, S. (2023) “Are emergent abilities of large language models a mirage?” *Advances in neural information processing systems 36 (NeurIPS 2023)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2304.15004>.
- Schmied, T. *et al.* (2026) “LLMs are greedy agents: Effects of RL fine-tuning on decision-making abilities,” *The fourteenth international conference on learning representations (ICLR 2026)*. OpenReview.net. Available at: <https://arxiv.org/abs/2504.16078>.
- Sciar, M. *et al.* (2024) “Quantifying language models’ sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting,” *The twelfth international conference on learning representations (ICLR 2024)*. OpenReview.net. Available at: <https://arxiv.org/abs/2310.11324>.
- Shao, Z. *et al.* (2024) “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300* [Preprint]. Available at: <https://arxiv.org/abs/2402.03300>.

- Shojaee, P. *et al.* (2025) “The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity,” *Advances in neural information processing systems 38 (NeurIPS 2025)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2506.06941>.
- Sinha, A. *et al.* (2026) “The illusion of diminishing returns: Measuring long horizon execution in LLMs,” *The fourteenth international conference on learning representations (ICLR 2026)*. OpenReview.net. Available at: <https://arxiv.org/abs/2509.09677>.
- Skean, O. *et al.* (2025) “Layer by layer: Uncovering hidden representations in language models,” *Proceedings of the 42nd international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 55854–55875. Available at: <https://arxiv.org/abs/2502.02013>.
- Snell, C. *et al.* (2025) “Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning,” *The thirteenth international conference on learning representations (ICLR 2025)*. OpenReview.net. Available at: <https://arxiv.org/abs/2408.03314>.
- Sutton, R.S. and Barto, A.G. (2018) *Reinforcement learning: An introduction*. Second. Cambridge, MA: MIT Press (Adaptive computation and machine learning).
- Szot, A. *et al.* (2024) “Large language models as generalizable policies for embodied tasks,” *The twelfth international conference on learning representations (ICLR 2024)*. OpenReview.net. Available at: <https://arxiv.org/abs/2310.17722>.
- Tam, Z.R. *et al.* (2024) “Let me speak freely? A study on the impact of format restrictions on large language model performance,” *Proceedings of the 2024 conference on empirical methods in natural language processing: Industry track*. Miami, Florida, US: Association for Computational Linguistics, pp. 1218–1236. Available at: <https://doi.org/10.18653/v1/2024.emnlp-industry.91>.
- Tan, W. *et al.* (2024) “True knowledge comes from practice: Aligning large language models with embodied environments via reinforcement learning,” *The twelfth international conference on learning representations (ICLR 2024)*. OpenReview.net. Available at: <https://arxiv.org/abs/2401.14151>.
- Team Olmo, Ettinger, A., *et al.* (2025) “Olmo 3,” *arXiv preprint arXiv:2512.13961* [Preprint]. Available at: <https://arxiv.org/abs/2512.13961>.
- TechCrunch (2026a) “A new kind of AI model from a ChatGPT inventor is thrilling developers.” TechCrunch, 18 September 2026, <https://techcrunch.com/2026/09/18/a-new-kind-of-ai-model-from-a-chatgpt-inventor-is-thrilling-developers/>.
- TechCrunch (2026b) “Amazon releases its own Jev clone as decision models flood the web.” TechCrunch, 1 October 2026, <https://techcrunch.com/2026/10/01/amazon-releases-its-own-jev-clone-as-decision-models-flood-the-web/>.
- The New Stack (2026) “TypeSafe launched Jev because sequential LLMs are ‘totally useless for computers’.” <https://thenewstack.io/typesafe-jev-system-one/>.
- Towers, M. *et al.* (2025) “Gymnasium: A standard interface for reinforcement learning environments,” *Advances in neural information processing systems 38 (NeurIPS 2025), datasets and benchmarks track*. Curran Associates, Inc. Available at: <https://doi.org/10.52202/085713-4916>.
- TypeSafe AI (2026a) “Choice: TypeSafe AI documentation.” <https://docs.typesafe.ai/primitives/choice>.
- TypeSafe AI (2026b) “Introducing System One models & Jev.” TypeSafe AI blog, <https://typesafe.ai/blog/introducing-system-one-models-and-jev>.
- TypeSafe AI (2026c) “System One: TypeSafe AI documentation.” <https://docs.typesafe.ai/concepts/system-one>.
- Walsh, E.P. *et al.* (2025) “2 OLMo 2 Furious (COLM’s version),” *Second conference on language modeling (COLM 2025)*. OpenReview.net. Available at: <https://arxiv.org/abs/2501.00656>.
- Wang, X. *et al.* (2024) “‘My answer is C’: First-token probabilities do not match text answers in instruction-tuned language models,” *Findings of the association for computational linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, pp. 7407–7416. Available at: <https://doi.org/10.18653/v1/2024.findings-acl.441>.
- Wei, J., Wang, X., *et al.* (2022) “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems 35 (NeurIPS 2022)*. Curran Associates, Inc. Available at: <https://doi.org/10.52202/068431-1800>.
- Wei, J., Tay, Y., *et al.* (2022) “Emergent abilities of large language models,” *Transactions on Machine Learning Research* [Preprint]. Available at: <https://openreview.net/forum?id=yzkSU5zdWd>.
- Wolf, T. *et al.* (2020) “Transformers: State-of-the-art natural language processing,” *Proceedings of the 2020 conference on empirical methods in natural language processing: System demonstrations*. Online: Association for Computational Linguistics, pp. 38–45. Available at: <https://doi.org/10.18653/v1/2020.emnlp-demos.6>.
- Wu, Y. *et al.* (2024) “SmartPlay: A benchmark for LLMs as intelligent agents,” *The twelfth international conference on learning representations (ICLR 2024)*. OpenReview.net. Available at: <https://arxiv.org/abs/2310.01557>.
- Yang, A. *et al.* (2024) “Qwen2.5 technical report,” *arXiv preprint arXiv:2412.15115* [Preprint]. Available at: <https://arxiv.org/abs/2412.15115>.

- Yang, A. *et al.* (2025) “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388* [Preprint]. Available at: <https://arxiv.org/abs/2505.09388>.
- Yao, S. *et al.* (2023) “ReAct: Synergizing reasoning and acting in language models,” *The eleventh international conference on learning representations (ICLR 2023)*. OpenReview.net. Available at: <https://arxiv.org/abs/2210.03629>.
- Zhao, T.Z. *et al.* (2021) “Calibrate before use: Improving few-shot performance of language models,” *Proceedings of the 38th international conference on machine learning*. PMLR (Proceedings of machine learning research), pp. 12697–12706. Available at: <https://arxiv.org/abs/2102.09690>.
- Zheng, C. *et al.* (2024) “Large language models are not robust multiple choice selectors,” *The twelfth international conference on learning representations (ICLR 2024)*. OpenReview.net. Available at: <https://arxiv.org/abs/2309.03882>.
- Zhou, C. *et al.* (2023) “LIMA: Less is more for alignment,” *Advances in neural information processing systems 36 (NeurIPS 2023)*. Curran Associates, Inc. Available at: <https://arxiv.org/abs/2305.11206>.